

IMU 指揮ジェスチャと発音予約同期による 分散合奏システム「タクトーン」の開発

情報工学科 2 年

25G1065

塩澤 匠生

—— チーム 23 ——

片岡 聖 (24G1038) 地曳 賢人 (24G1175)

御代川 稜 (24G1131) 梅澤 颯太 (25G1021)

齋藤 翔太 (25G1053)

2026 年 7 月 14 日

1 はじめに

合奏は複数の奏者が呼吸を合わせて一つの音楽を作る体験であるが、楽器の習熟には長い訓練を要し、経験のない人が合奏の中心である指揮者の役割を体験する機会は少ない。楽器の習熟を前提としない音楽体験を提供する試みはこれまでにも行われており、大きく二つに分類できる。一つはセンサで身体動作を捉えて発音に変換する電子楽器であり、距離センサ等を用いた非接触型電子楽器の開発 [1] が報告されている。もう一つは家庭用ゲーム機でコントローラを振る動作により楽器演奏を模擬するもので、市販ゲーム [2] が代表例である。これらはいずれも一人の操作を一つの音に変換するものが中心であり、「複数の独立した楽器を一人の指揮で同時に動かす」という合奏の構図そのものを体験させるシステムは少ない。また、ネットワークを介して複数拠点の奏者を接続する遠隔合奏の研究 [3] はあるが、参加者全員が演奏技術を持つことを前提としており、未経験者が指揮者として合奏全体を駆動する構成は見当たらない。

そこで本プロジェクトでは、指揮者の腕の振りを慣性計測装置（IMU：Inertial Measurement Unit）で検出し、無線で接続された複数の楽器ノードを同時に演奏させる分散合奏システム「タクトーン」を製作した。システムは指揮者ノード 1 台（ESP32-S3）と楽器ノード 5 台（Arduino UNO R4 WiFi）、および音色合成を担う PC 5 台から成り、指揮者ノードが検出した拍を無線 LAN 上の UDP マルチキャストで全楽器ノードへ配信する。

このようなシステムの技術的課題は、無線通信の遅延ゆらぎ（ジッタ）の下で複数楽器の発音時刻をそろえることである。合奏のタイミングに関する心理学的研究の総説 [4] によれば、プロの合奏でも演奏者間の発音時刻のずれは 30～50 ms 程度観察され、数十 ms のずれであれば主観的にはうまく同期しているとみなされる。本プロジェクトではチームでの議論を経て、これに対して安全側の目標として楽器間同期誤差 20 ms 以内を掲げた。また遠隔合奏の先行研究 [3] でも通信遅延の管理が主要な設計課題とされており、本システムでも遅延そのものを消すのではなく、遅延の影響を打ち消す設計が必要となった。

報告者はチーム内で Arduino 系プログラム全般、すなわち指揮者ノード・楽器ノード・共通ライブラリ層の設計・実装と、性能検証の全体を担当した。実装した主な機能は、(1) IMU による拍検出とテンポ・強弱の推定、(2) 自作バイナリプロトコルによる UDP マルチキャスト配信、(3) 時計同期と発音予約による楽器間同期、(4) 拍番号に基づく楽譜進行と輪唱制御である。音色合成（PC 側）と楽譜データはチームの他メンバーが実装し、報告者が設計した発音指示パケットの仕様が両者の境界インタフェースとなった。

評価では、計画書で定めた 9 項目の性能指標（MOP：Measure of Performance）を全項目実測した。7 項目が目標を達成し、楽器間の発音時刻差は中央値 7 ms（目標 20 ms の約 3 分の 1）であった。一方、指揮から楽器への通信遅延（MOP5、目標 30 ms 以内）は、アクセスポイントのマルチキャスト配送が約 205 ms 周期のバースト状になるという想定外の実挙動により、実測平均約 100 ms で目標を達成できなかった。ただし、この遅延は発音予約方式の再設計（先読み時間 45→220 ms・時計同期の min フィルタ化）によって演奏品質への影響をほぼ吸収できることを実測

で確認し、予約に間に合わなかった拍は全体の 3.1%まで抑えられた。さらに成果発表会では聴衆 43 名によるエンタテインメント性評価を実施し、「楽しさ・高揚感」が 7 段階中平均 6.21 という結果を得た。

本システムの特徴は、安価なマイコンと標準的な無線 LAN のみで「届くのはバラバラでも、鳴るのは同時」という分散合奏を成立させた点にある。計画時の想定と最終実装の相違点、および未達成となった指標の原因分析についても、本報告書で数値に基づいて明示する。

2 作成したシステムの概要

2.1 システム全体の構成

システム構成を図 1 に示す。システムは次の 3 種類の要素から成る。

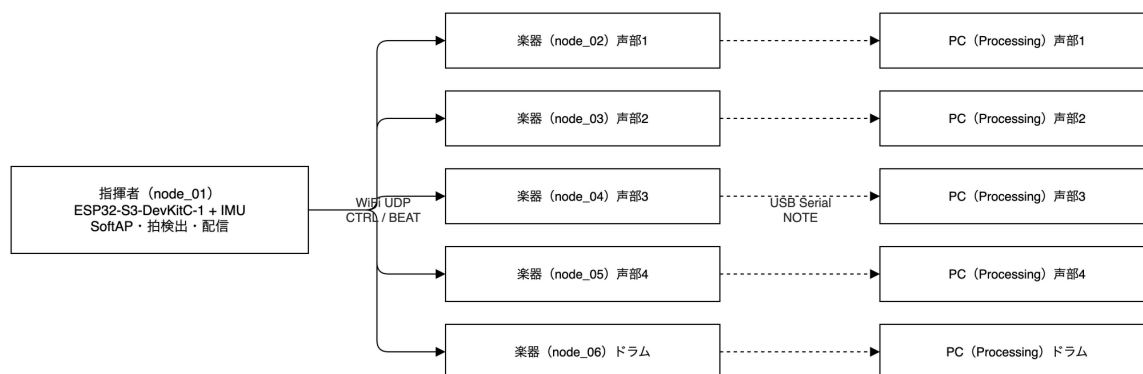


図 1 システム全体構成 (指揮者 1 台・楽器 5 台・PC 5 台)

指揮者ノード (node_01) ESP32-S3-DevKitC-1 に IMU (GY-521, MPU6050) を I2C 接続したマイコンである。自身が無線 LAN アクセスポイント (SoftAP) を起動し、IMU の加速度から拍を検出して、テンポ・強弱を含む制御パケット CTRL と拍パケット BEAT を UDP マルチキャスト (239.0.0.1:5001) で全楽器ノードへ配信する。

楽器ノード (node_02~06) Arduino UNO R4 WiFi[5] 5 台である。4 声部 (トランペット・ホルン・フルート・オルガンの音色を割当) とドラム 1 台から成り、各ノードが自パートの楽譜データを保持する。BEAT の予約時刻に合わせて楽譜を 1 拍進め、発音指示パケット NOTE を USB シリアル (115200 bps) で自分専用の PC へ送る。

PC (5 台) Processing[6] 製の音色合成アプリケーションが NOTE を受信し、楽器定義 JSON に基づく倍音加算合成で発音する。PC は楽器ごとに 1 台を割り当て、スピーカも独立させることで「楽器が別々の場所で鳴る」合奏の空間性を再現する。

通信方式には UDP マルチキャストを採用した。TCP は再送による遅延変動が拍の同期に不向きであること、1 対 5 の同報にはマルチキャストが帯域・実装の両面で単純であることが理由である。再送がないことによる取りこぼしは、後述の同一パケット 4 連送と、拍番号に基づく楽譜進行の自己復帰（2.6 節）で補償する。また無線 LAN は指揮者ノード自身の SoftAP で構築し、会場のネットワーク設備に依存せずシステム単体で動作する構成とした。

2.2 動作モード

システムは自由演奏モードとゲームモードの 2 モードを持つ。自由演奏モードでは指揮者の振りがそのまま演奏テンポになる。ゲームモードでは目標テンポの提示とテンポ維持精度の採点を行う。モード選択は指揮者ノードの IMU ナビゲーション（腕の左右振りでカーソル移動・縦振りで決定）で行い、選択状態は CTRL パケットの予約領域を用いて全ノードへ伝達される。本報告書は報告者の担当である同期・通信・拍検出を中心に述べるため、ゲームモードの採点詳細は付録のソースコードに委ねる。

2.3 IMU による拍検出の原理

指揮者ノードは腕の「振り下ろし」を拍として検出する。検出器の全体構造を図 2 に示す。IMU の 3 軸加速度 $\mathbf{a}[n]$ （単位 g，サンプリング周期約 5 ms）に対し、まず 1 次 IIR ローパスフィルタでノイズを抑える。

$$\mathbf{a}_{\text{lpf}}[n] = (1 - \alpha) \mathbf{a}_{\text{lpf}}[n - 1] + \alpha \mathbf{a}[n], \quad \alpha = 0.10 \quad (1)$$

ここで n はサンプル番号である。次に、起動時キャリブレーション（2 s 間の静止計測）で求めた静止時ノルム g_0 （ $\approx 1 \text{ g}$ ，重力に相当）を差し引いた動加速度ノルム $d[n]$ を求める。

$$d[n] = \|\mathbf{a}_{\text{lpf}}[n]\| - g_0 \quad (2)$$

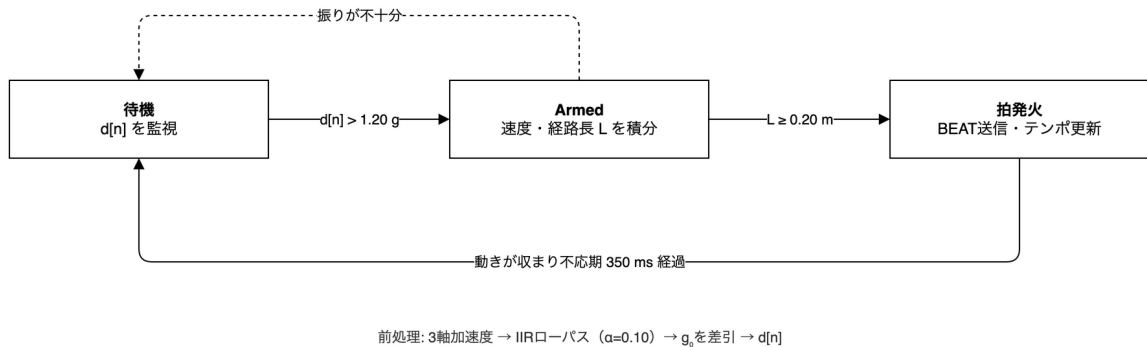


図 2 拍検出のゲート式状態機械（しきい値・経路長・不応期の 3 段構え）

拍検出はしきい値の単純比較ではなく、「腕が実際に動いた距離」を測るゲート式の状態機械とした。 $d[n] > 1.20 \text{ g}$ で武装 (Armed) 状態に入り, Armed 中のみ動加加速度を二重積分して移動経路長を推定する。重力加速度を $g_a = 9.80665 \text{ m/s}^2$, サンプル間隔を Δt , 動加加速度ベクトルを $\mathbf{a}_{\text{dyn}}[n]$ (単位 g) とすると, 速度 $\mathbf{v}$ と経路長 L は次で更新される。

$$\mathbf{v}[n] = \mathbf{v}[n-1] + g_a \mathbf{a}_{\text{dyn}}[n] \Delta t, \quad L[n] = L[n-1] + \|\mathbf{v}[n]\| \Delta t \quad (3)$$

$L[n] \geq 0.20 \text{ m}$ に達した瞬間に拍を発火する。発火後も Armed を維持し, 動加加速度が収まる ($d[n] < 0.20 \text{ g}$, またはピーク値の 40%未満が 40 ms 継続) までは再突入させないことで, 1 回の振りから 1 拍のみを発火させる。さらに不応期 350 ms (約 170 BPM の拍間隔に相当) を併用し, フィルタの尾引きによる二重発火を構造的に防ぐ。しきい値の単純比較ではなく経路長を併用したのは, しきい値だけでは振幅の小さい速い揺れも拍として拾いうるため, 「実際に腕が一定距離動いた」動作だけを選別する目的である。これらのパラメータの決定経緯は 3.3 節で述べる。

テンポは拍間隔 $\Delta\tau_k$ (ms) から瞬時値 $b_k = 60000/\Delta\tau_k$ を求め, 指数移動平均 (EMA: Exponential Moving Average) で平滑化する。

$$\bar{b}_k = (1 - \beta) \bar{b}_{k-1} + \beta b_k, \quad \beta = 0.30 \quad (4)$$

ここで k は拍番号であり, $\bar{b}_k$ は 40~240 BPM に制限して CTRL で配信する。強弱は振りの動加加速度ピークの大きさを velocity 値 (0~127) へ写像して CTRL に載せ, 楽器側で楽譜の velocity と乗算合成される。

2.4 発音予約と時計同期の原理

本システムの同期設計の核心は, 「パケットの到着時刻に頼らず, 発音予約と時計同期で発音時刻をそろえる」ことである。図 3 にこの動作を示す。指揮者は拍を検出すると, マスタ時計 (指揮者の `millis()`) で $t_{\text{lookahead}} = 220 \text{ ms}$ 先の未来時刻を発音予約時刻 `playAtMasterMs` として BEAT に載せて送る。各楽器はマスタ時計との時計オフセットを推定しておき, 予約時刻が来るまで待ってから発音する。こうすることで, 配送遅延が拍ごと・楽器ごとにばらついていても, 予約時刻までに届きさえすれば全楽器の発音はマスタ時計基準でそろえる。

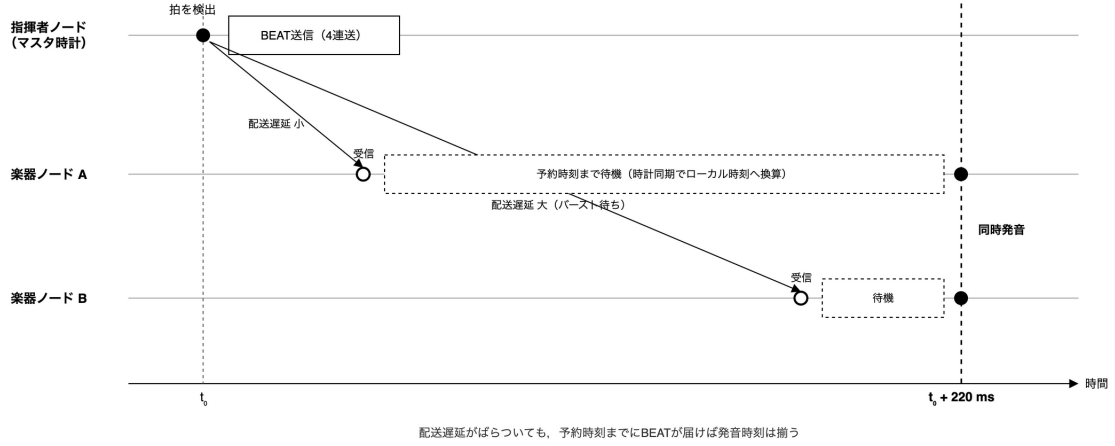


図3 発音予約と時計同期による同期の原理（配送遅延のばらつきを待ち時間が吸収する）

時計オフセットの推定は次のとおりである．パケット i の送信時マスタ時刻を T_i （ヘッダの timestampMs），受信時のローカル時刻を t_i とすると，観測値

$$s_i = T_i - t_i = \theta - D_i \quad (5)$$

が得られる．ここで θ は真の時計オフセット， $D_i \geq 0$ は配送遅延である． s_i は真値 θ から配送遅延のぶんだけ必ず下側に外れるため，直近の時間窓 W （2000 ms）内の最大値を推定値として採用する．

$$\hat{\theta} = \max_{i \in W} s_i \quad (6)$$

これは「窓内で最も配送遅延が小さかったサンプルだけを信じる」min フィルタであり，NTPv4 のクロックフィルタが遅延の小さい観測を優先する設計 [7] と同じ考え方である．楽器は発火判定のたびに予約時刻をローカル時刻 $t_{\text{fire}} = \text{playAtMasterMs} - \hat{\theta}$ へ変換し，制御周期 2 ms で到達を判定する．発火判定の中核部をリスト 1 に示す（全体は付録リスト 5）．

リスト 1 楽器ノードの発音予約発火判定（抜粋）

```

1  const int32_t targetLocalMs =
2      (int32_t)data.receiver.pending.playAtMasterMs - data.sync.offsetMs;
3  const int32_t waitMs = targetLocalMs - (int32_t)now;
4  if (waitMs <= 0) {
5      fired          = true;    // 予約時刻に到達：この周期で発音する
6      firedBeatNo = data.receiver.pending.beatNo;
7      data.receiver.pending.valid = false;
8  }
9  // waitMs > 0: 次ループ（2ms 後）で再評価する

```

式 (6) の min フィルタの実装をリスト 2 に示す。遅延のより小さいサンプルは即時採用して推定を引き上げる一方、窓満了時に窓内最大値へ「引き直す」ことで、時計のドリフト（楽器側の時計が速い個体）にも窓長ぶんの遅れで追従できるようにした点が実装上の工夫である。

リスト 2 時計オフセットの min フィルタ（抜粋）

```

1  const int32_t sample = (int32_t)(timestampMs - millis());
2  if (sample > data.sync.offsetMs) {
3      data.sync.offsetMs = sample;    // 遅延のより小さいサンプルは即採用
4  }
5  if (!winValid_) {                  // 2000ms 窓の開始
6      winValid_ = true;  winStartMs_ = nowMs;  winMaxSample_ = sample;
7  } else {
8      if (sample > winMaxSample_) winMaxSample_ = sample;
9      if (nowMs - winStartMs_ >= cfg_.clockSyncWindowMs) {
10         data.sync.offsetMs = winMaxSample_; // 窓満了：窓内最大へ引き直し
11         winValid_ = false;
12     }
13 }

```

なお開発当初の実装では、式 (6) の代わりに EMA でオフセットを平滑化しており、 $t_{\text{lookahead}}$ も 45 ms であった。これらを変更した経緯と理由は 3.3.3 節および 5.2 節で述べる。また指揮者ノードの再起動でマスタ時計が巻き戻った場合は、 $|s_i - \hat{\theta}|$ が 1000 ms を超えた時点でフィルタを介さず即時追従（スナップ）し、発音予約を破棄して新しいマスタ時計で再開する。

2.5 通信プロトコル

ノード間の通信は表 1 に示す 3 種類（+ UI 中継 1 種類）の自作バイナリパケットで行う。全パケットを 20 バイト固定長（共通ヘッダ 12 バイト＋ペイロード 8 バイト、リトルエンディアン）に統一し、先頭 2 バイトのマジックナンバー 0x4F52（ASCII “OR”）で境界の再同期を可能にした。固定長・単純構造としたのは、Arduino UNO R4 のメモリと処理時間の制約下で、受信処理を制御周期 2 ms 内に確実に収めるためである。パケット定義は付録のリスト 3 に示す。

表 1 通信パケットの種類と役割

種別	経路	頻度	主な内容
CTRL	指揮者 → 全ノード (UDP)	20 Hz	テンポ (BPM×8), 強弱, 状態, モード
BEAT	指揮者 → 全ノード (UDP)	拍ごと (4 連送)	拍番号, 発音予約時刻
NOTE	楽器 → PC (USB シリアル)	発音ごと	音高, 強弱, 音長, パート, 楽器番号
UI	楽器 → PC (USB シリアル)	変化時 (最大 5 Hz)	画面状態の中継 (ゲームモード用)

信頼性設計として、BEAT は同一内容を 4 連送する。UDP マルチキャストには再送がなく、拍の取りこぼしは演奏の欠落に直結するためである。同一拍番号の重複受信は楽器側で排除する。この設計の効果は評価で確認され、最終構成でのパケット欠落は 5 ノードすべてで 0.0% であった (4.10 節)。

2.6 楽譜進行と輪唱

楽譜は各楽器ノードが `ScoreEvent` 配列（拍位置・ノート番号・強弱・音長）として保持し、発火した拍の指揮者拍番号 `firedBeatNo` から楽譜位置を直接算出する。経過時間ではなく拍番号で楽譜を引くため、パケットを1拍取りこぼしても次の拍で正しい位置に自己復帰し、ずれが蓄積しない。この「外部からの拍イベントに楽譜位置を追従させる」構成は、演奏の弾き飛ばしに追従する楽譜追跡研究 [8] の考え方を単純化して取り入れたものである。輪唱はパートごとの開始遅延 `headRestBeats` と全パート共通のサイクル窓（曲長+最終声部の遅延）で管理し、最終声部が1周を終えるまで先頭声部が次の周回を始めない。

2.7 PC 側の音色合成

PC 側の音色合成アプリケーション（Processing）はチームメンバー（梅澤・齋藤）の担当であるが、システム全体の動作理解のため概要を述べる。アプリケーションは楽器ノードから USB シリアルで届く NOTE を受信し、音高・強弱・音長に従って発音する。音色は倍音加算合成（基音の整数倍の正弦波を振幅比で重ねる方式）で生成され、倍音構成・エンベロープ・ビブラート等のパラメータは楽器ごとの JSON ファイルに外部化されている。この JSON は実楽器の録音を周波数分析して作成されたものであり、楽器を差し替える際もファームウェアの変更が不要である。NOTE パケットの仕様（20 バイト固定・楽器番号の割当）が報告者の担当したファームウェアと PC アプリケーションの境界インタフェースであり、チーム内の分担境界と一致させることで並行開発を可能にした。

3 システム実装

3.1 チーム構成と役割分担

チーム 6 名の役割分担を表 2 に示す。報告者は Arduino 系プログラム全般、すなわち指揮者ノード・楽器ノード・共通ライブラリ層の設計と実装、および本報告書で述べる性能検証の全体を担当した。齋藤が楽譜データと音色の周波数分析を、梅澤が Processing による音色合成・UI を担当し、報告者が設計した NOTE パケット仕様（2.5 節）がファームウェアと PC アプリの境界インタフェースとなった。

表 2 チーム 23 の役割分担

メンバー	学籍番号	主な担当
塩澤 匠生	25G1065	Arduino 系プログラム全般（指揮者・楽器・共通層）、性能検証
齋藤 翔太	25G1053	楽譜データ作成・音階生成ロジック・周波数分析
梅澤 颯太	25G1021	Processing による音色合成・演奏再生・UI
御代川 稜	24G1131	議事録（持ち回り 1 番手）、計画書編纂
地曳 賢人	24G1175	議事録（2 番手）、スケジュール管理
片岡 聖	24G1038	議事録（3 番手）、WBS・役割整理、機材調達

ファームウェアの構造は全ノード共通で Embedded-Module-Architecture (EMA) [9] を採用した。EMA は報告者が本プロジェクト以前から整備してきた組込み向けの設計様式であり、図 4 に示すようにループを入力・ロジック・出力の 3 フェーズに分け、全モジュールが共有構造体 `SystemData` のみを介して通信する。モジュール同士の直接呼び出しを禁止することで、(1) 拍検出・通信・LED 表示といった関心事を独立に差し替えられる、(2) 指揮者と楽器で通信・LED 等のモジュールを共通ライブラリとして共有できる、(3) ロジック部が `SystemData` の読み書きだけで完結するため実機なしの机上検証がしやすい、という利点がある。実際、共通ライブラリ層（プロトコル定義・UDP 送受信・受信処理・NOTE 送出・LED・デバッグ出力）は 7 ノードのファームウェアから完全に共有された。

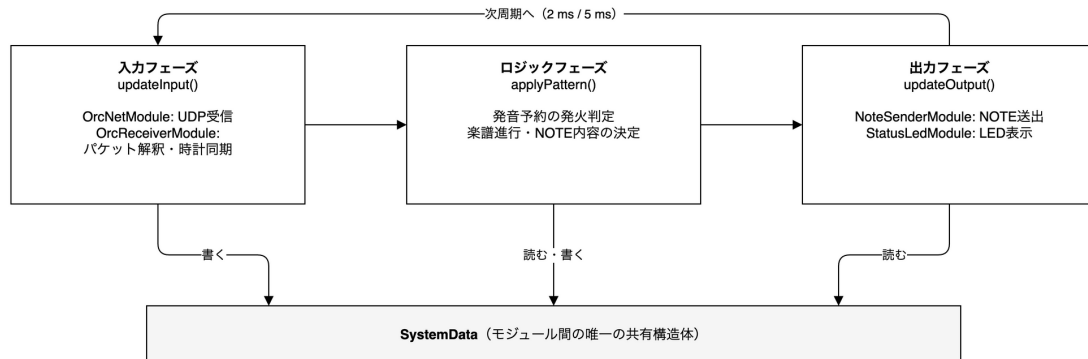


図 4 Embedded-Module-Architecture の 3 フェーズループと共有構造体

ファームウェアの主要な処理は、以下の関数によって構成される。

loop() 関数 入力・ロジック・出力の 3 フェーズを順に実行し、制御周期（楽器 2 ms・指揮者 5 ms）で繰り返す全ノード共通のメインループである。フローチャートを図 5 に示す。

applyPattern() 関数（指揮者） IMU の加速度から拍を検出し、テンポ・強弱を推定して BEAT・CTRL の送信内容を決定する（2.3 節）。フローチャートを付録の図 A.20 に示す。

OrcReceiverModule::updateInput() 関数（楽器） UDP パケットの受信・検証、時計オフセットの min フィルタ更新、発音予約の登録を行う（2.4 節）。フローチャートを図 A.21 に示す。

applyPattern() 関数（楽器） 発音予約の発火判定と楽譜進行・輪唱制御を行い，NOTE の出力要求を生成する（2.6 節）．フローチャートを図 A.22 に示す．

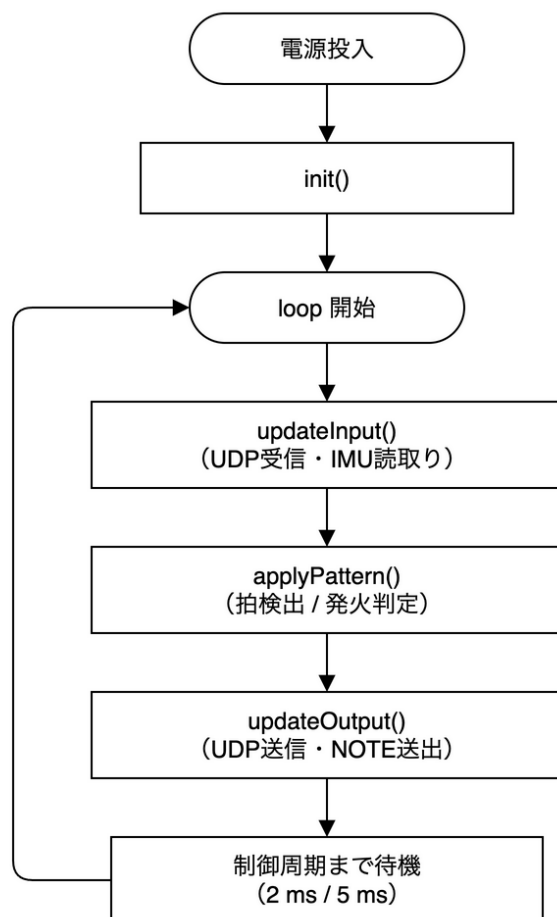


図 5 loop() 関数（メインループ）のフローチャート

計画時のスケジュールを図 6 に示す．設計（WBS 110～120，全員で分担し報告者はアーキテクチャ・通信設計を担当）・製造（210～260，報告者は共通層・指揮者・楽器ノードを担当）・テスト（310～330，報告者）・発表準備の 4 フェーズ構成である．実績はおおむねこの計画に沿ったが，成果発表会が台風の影響で 7 月 1 日から 7 月 8 日へ延期されたことに伴い，最終週に性能検証の再実施と同期対策（3.3.3 節・5.2 節）を追加で行った点が計画との相違である．



図 6 計画時のスケジュール (WBS タスク別, 2026 年 4 月 15 日～7 月 8 日)

3.2 報告者の担当範囲と技術的課題

報告者は前節のとおり Arduino 系プログラム全般と性能検証を担当した。この担当範囲を実現するためには、次の 4 つの技術的課題を解決する必要がある。

1. 1 回の腕の振りから過不足なく 1 拍だけを検出すること (振りかぶり・フィルタの尾引きによる二重検出の防止)
2. 無線 LAN の遅延ゆらぎの下で 5 台の楽器の発音時刻を 20 ms 以内にそろえること
3. UDP マルチキャストのパケット取りこぼしが演奏の破綻につながらないこと
4. 7 個のファームウェアプロジェクト (指揮者 2 種 + 楽器 5 台) を 1 人で並行保守し、性能を定量検証できる体制を作ること

課題 (1) には 2.3 節のゲート式状態機械で対処し、パラメータは次節の予備実験で決定した。課題 (2) には 2.4 節の発音予約方式で対処したが、開発終盤の実測でアクセスポイントの配送特性に起因する想定外の問題が判明し、パラメータと推定方式を再設計した (3.3.3 節・5.1 節)。課題 (3) には BEAT の 4 連送と拍番号ベースの楽譜進行で対処した。課題 (4) には共通ライブラリ化 (EMA) と、ビルドフラグで計測コードを組み込む検証環境 (4.1 節) で対処した。

3.3 予備実験とパラメータ決定

3.3.1 拍検出パラメータ

拍検出の 3 つのパラメータ (武装しきい値 1.20 g・経路長しきい値 0.20 m・不応期 350 ms) は、実機で指揮動作を繰り返しながら段階的に決定した。武装しきい値は、楽譜台に腕を置く程度の動

作（約 0.5 g）では反応せず、意図した振り下ろし（1.5 g 以上）を確実に捉える中間値として 1.20 g を選んだ。経路長しきい値は、手首だけの小さな揺れ（経路長 0.1 m 未満）を棄却しつつ肘からの振り（0.3 m 以上）を全て通す値として 0.20 m とした。不応期は、検出対象とする最高テンポ 170 BPM（拍間隔 353 ms）を上限に 350 ms と定め、これにより 1 回の振りの往復動作（振り下ろし＋戻し）が 2 拍と誤検出されることを構造的に防いだ。

テンポ平滑化係数 $\beta = 0.30$ （式 (4)）は、「1 拍の乱れで演奏テンポが跳ねない」平滑の強さと「テンポ変化への追従の速さ」の折衷として、実機で振り心地を確かめながら決定した。この値でのテンポ追従の遅れは最大 1.2 拍であり、目標（2 拍以内）に収まることを評価で確認した（4.7 節）。

3.3.2 制御周期

楽器ノードの制御周期は当初 5 ms であったが、2 ms へ短縮した。発音予約方式では発火判定の時間分解能がそのまま同期誤差の下限になるためである。短縮にあたっては処理が周期内に収まるかが問題になるため、3 フェーズそれぞれの処理時間を実測した。結果を表 3 に示す。最も重い入力フェーズでも最大 0.72 ms、3 フェーズ合計でも最大 1.50 ms であり、2 ms 周期に対して余裕があることを確認して採用した（計測の詳細は 4.9 節）。

表 3 3 フェーズの処理時間の実測（12,326 サンプル）

フェーズ	平均	最大
入力（IMU 読取り・UDP 受信）	71 μ s	724 μ s
ロジック（applyPattern）	4 μ s	78 μ s
出力（UDP 送信・LED）	15 μ s	912 μ s
合計	90 μ s	1,499 μ s

3.3.3 発音予約の先読み時間と時計同期方式

発音予約の先読み時間 $t_{\text{lookahead}}$ は、当初、見かけの配送遅延（開発初期の計測で数～数十 ms）に余裕を持たせた 45 ms としていた。しかし開発終盤にデバイス側計測による検証環境を整備して測り直した結果、配送遅延は想定と大きく異なっていた。表 4 に、この再計測（2026 年 7 月 10 日、1024 受信）によるノード別の BEAT 配送遅延を示す。全 5 ノードで遅延の中央値は 95～99 ms・最大は 198～209 ms に達し、一様分布 $[0, 204.8]$ ms の理論値（中央値 102 ms・95 パーセンタイル 195 ms）とよく一致した。これは SoftAP のマルチキャスト配送が約 204.8 ms 周期のバースト状であることを意味し（原因の特定は 5.1 節）、45 ms の先読みでは 45.4 % の拍で BEAT が予約時刻までに届かないことも判明した。

表 4 先読み時間の再設計の根拠としたノード別 BEAT 配送遅延の実測 (2026 年 7 月 10 日)

ノード	中央値	平均	95 パーセンタイル	最大
node_02	95 ms	97 ms	185 ms	198 ms
node_03	99 ms	103 ms	191 ms	209 ms
node_04	96 ms	98 ms	186 ms	202 ms
node_05	95 ms	97 ms	184 ms	199 ms
node_06	97 ms	101 ms	189 ms	202 ms
一様分布 [0, 204.8] ms の理論値	102 ms	102 ms	195 ms	205 ms

この実測に基づき、先読み時間を「バースト周期 204.8 ms + 余裕」として 220 ms へ拡大した。バースト 1 周期を確実に覆う最小の切りのよい値であり、これより短いと届かない拍が残り、これより長くしても同時性は改善せず指揮への追従遅れだけが増えるためである。同時に、時計オフセットの推定を当初の EMA 平滑化から式 (6) の min フィルタへ変更した。バースト環境では EMA は真値より約 40~55 ms 遅れた位置に収束し、発音時刻の遅れと計測値の過小表示を招いていたためである。これらの変更の効果の定量評価は 5.2 節で詳述する。

計画時の想定と最終実装の主な相違点を表 5 にまとめる。いずれも開発中の実測または実機テストで判明した事実に基づく変更である。

表 5 計画時の想定と最終実装の相違点

項目	計画時	最終実装
発音予約の先読み時間	45 ms	220 ms (5.2 節)
時計オフセット推定	EMA ($\alpha = 0.10$)	min フィルタ (窓 2000 ms)
MOP2 (音階誤差) の測定法	実音録音の周波数分析	合成アルゴリズムの忠実再現 (4.3 節)
IMU 喪失時の Fallback 遷移	自動で内蔵テンポに移行	自動遷移を無効化 (3.4 節)
成果発表会	2026 年 7 月 1 日	2026 年 7 月 8 日 (台風で延期)

3.4 担当部分以外の実装と工夫

指揮者ノードの回路図を図 7 に、使用した主要部品を表 6 に示す。指揮者ノードは ESP32-S3-DevKitC-1 と IMU (GY-521) を 10 ピンコネクタ経由で I2C 接続した構成であり、AD0 端子を GND へ接続して I2C アドレスを 0x68 に固定した。開発初期には XIAO ESP32-S3 Sense を指揮者に用いたが、外付け IPEX アンテナの接触不良に起因する UDP パケットロスが多発したため、PCB 内蔵アンテナの DevKitC-1 へ切り替えた。これによりパケットロスは大幅に改善し、以降の評価はすべて DevKitC-1 で実施した。楽器ノードは Arduino UNO R4 WiFi 単体で外部配線がなく、USB ケーブル 1 本で PC と接続する。部品点数を最小化したのは、発表会で 5 台の楽器を確実に設営・撤収するためであり、配線起因のトラブルを構造的に排除する意図である。

3.5 開発支援ツールの利用と検証

3.5.1 ビルド環境：PlatformIO

ファームウェアのビルド環境には Arduino IDE ではなく PlatformIO[10] を採用した。採用理由は次の 4 点であり、いずれも本プロジェクトで実際に効果があったものである。第一に、設定ファイル `platformio.ini` にプラットフォームのバージョン (`espressif32@6.10.0`) とボード定義を宣言的に固定できるため、チームメンバーの誰がビルドしても同一のツールチェーンが再現される。Arduino IDE ではボードパッケージのバージョンが GUI 操作に依存し、環境差異の混入を防げない。第二に、コマンドラインでビルドできるため、GitHub Actions による CI (継続的インテグレーション) に統合でき、`firmware/` 配下の変更をプッシュするたびに全 7 プロジェクトが自動ビルドされる体制を構築できた。これにより「あるノードの修正が別ノードの共通ライブラリを壊す」退行を、実機に書き込む前に検出できた。第三に、本システムは指揮者 2 種 + 楽器 5 台の計 7 プロジェクトから成るが、プロジェクトごとの依存関係とビルドフラグを `platformio.ini` で個別管理しつつ、共通ライブラリ (`common/lib/`) をシンボリックリンクなしで共有できた。第四に、ビルドフラグによる条件コンパイルの切替 (デバッグ出力の `SERIAL_DEBUG`, 性能計測コードの `MOP_TEST`) を設定ファイルの 1 行で行えるため、計測用ファームと本番ファームを同一ソースから確実に生成できた。シリアルモニタ (`pio device monitor`) がビルドと同じ CLI から使えることも、5 台同時のログ収集 (4.1 節) と相性がよかった。

3.5.2 生成 AI の利用と検証

開発には生成 AI (Anthropic Claude, コーディングエージェント Claude Code) を利用した。利用目的は (1) EMA に基づくモジュール雛形と共通ライブラリのコード生成, (2) 実装のレビュー (バッファ境界・整数オーバーフロー等の指摘), (3) 検証環境 (シリアル同時収集・MOP 自動集計の Python スクリプト) の作成, (4) 調査レポートやドキュメントの整備である。技術情報源としては Arduino 公式ドキュメント [5] と Processing 公式リファレンス [6] を併用した。

生成 AI の出力は無検証では採用せず、次の規律を機械的に適用した。第一に、「実機で未検証のコードに AI 起点の変更を積まない」ことをチームのルールとして明文化し、生成コードは必ずビルド・実機書き込み・動作確認を経てから採用した。これは開発初期に、机上では正しく見える拍検出の修正が実機では挙動を悪化させた経験から定めたものである。第二に、ロジックの正否は可能な限り実機によらない再現検証で確かめた。例えば輪唱進行ロジックは Python 上でサイクル窓の挙動を再現する検証スクリプトを作成し、120 拍分・20 項目の検査で「ファームウェアにバグなし・問題は実機側の構成」と切り分けた。第三に、AI の提案自体を実測で棄却した例がある。同期改善策として AI が提案した 3 案 (先読み時間の拡大・ビーコン間隔の短縮・min フィルタ化) のうち、ビーコン間隔の短縮は実測でバースト周期を 1 ms も変えないことを確認して撤去し、効果が実証された 2 案のみを最終構成に残した (5.1 節)。また計測系についても、AI が作成した集計スクリプトの公表値を別経路の手計算・独立集計で検算し、計測方式の不整合 (受信時刻と発火

時刻の混同)を発見して計測パイプラインごと再構築した。

以上のとおり、生成 AI は雛形生成と分析の加速手段として利用し、設計判断・パラメータ決定・最終的な採否はすべて実測に基づいて報告者が行った。採用したコードの内容は報告者が説明できる状態を維持しており、本報告書の記述と付録のソースコードはその実証である。このほか、構成図・フローチャートの作図に draw.io を、実測データの集計・グラフ化に Python (NumPy・Matplotlib) を使用した。

4 評価方法と結果

4.1 評価方法

計画書で定めた 9 項目の性能指標 (MOP) を全項目実測で評価した。評価の目的は次の 3 点を確認することである。

1. 指揮者の動作が正しく拍・テンポとして認識されるか (MOP1・MOP6)
2. 5 台の楽器が音楽として成立する精度で同時に・正しく発音するか (MOP2・MOP3・MOP4・MOP5・MOP9)
3. システムが実用に足る応答性・負荷で動作するか (MOP7・MOP8)

測定にあたり、ファームウェアに MOP_TEST コンパイルフラグで各指標専用の計測ログ出力を組み込み、5 台の楽器ノードのシリアル出力を Python スクリプト (tools/verification/scripts/) で同時収集・自動集計する検証環境を報告者が構築した。同期系 (MOP4・MOP5) は USB シリアルの到着ジッタが混入しないデバイス側計測 (BEAT 受信時と発音発火時に、そのときの推定マスタ時刻を 1 行記録する方式) で測定し、2026 年 7 月 10~11 日に 3 回 (対策前・対策後・最終構成、延べ 560 拍 × 5 ノード) の実測を行った。このとき各楽器ノードが記録する観測項目を表 7 に示す。機能系 (MOP1・MOP6・MOP7・MOP8) は 7 月 2 日に、MOP3・MOP9 は 7 月 9 日に測定した。各測定の生データ・集計結果はチームリポジトリの tools/verification/results/ 配下に保存しており、以降の各節にファイルパスを明記する。

表 7 同期系計測 (MOP4・MOP5) で記録する観測項目と詳細

観測項目	詳細
種別	R = BEAT 受信時, F = 発音発火時 (各拍で 2 行記録される)
beatNo	指揮者が振った拍の通し番号
partId	記録した楽器ノードの識別子 (0x02~0x06)
playAtMasterMs	BEAT に格納された発音予約時刻 (マスタ時計)
deviceMs	受信/発火した瞬間のローカル時刻 (millis())
offsetMs	その時点の推定時計オフセット $\hat{\theta}$
localMasterMs	推定マスタ時刻 ($\text{deviceMs} + \hat{\theta}$). ノード間の発音時刻差 (MOP4) はこの値の同一拍でのレンジで求める
lateMs	予約時刻に対する遅刻 $\max(0, \text{localMasterMs} - \text{playAtMasterMs})$

全 9 項目の目標値と実測値の一覧を表 8 に示す. 7 項目が目標を達成し, MOP4 は判定基準 (最大値) では未達だが中央値は目標の約 3 分の 1, MOP5 は未達であった. 以降, 各項目の測定方法と結果を個別に述べ, 未達項目の原因分析と対策は第 5 章で論じる.

表 8 性能指標 (MOP) の目標値と実測値の一覧

指標	目標値	実測値 (最終構成)	判定
MOP1 拍検出の正確性	正解率 $\geq 90\%$	100.0% (120 拍/120 拍)	達成
MOP2 音階の誤差	平均 $< 3.6 \text{ cent}$	平均 0.737 cent・最大 1.907 cent	達成
MOP3 楽譜との相違	誤ノート 0 件	0 件	達成
MOP4 楽器間同期誤差	$\leq 20 \text{ ms}$	中央値 7 ms・最大 65 ms	未達
MOP5 指揮 → 楽器の通信遅延	$\leq 30 \text{ ms}$	平均 100 ms・最大 215 ms	未達
MOP6 テンポ追従の遅延	$\leq 2 \text{ 拍}$	最大 1.2 拍	達成
MOP7 起動時間	$\leq 5 \text{ s}$	最大 2.3 s	達成
MOP8 CPU 負荷 (入力フェーズ)	$\leq 2 \text{ ms}$	最大 0.72 ms	達成
MOP9 パケットロス耐性	欠落 $\leq 5\%$	全ノード 0.0%	達成

また性能指標とは別に, 成果発表会 (2026 年 7 月 8 日) でシステムのエンタテインメント性に関する聴衆アンケートを実施した. 方法と結果は 4.11 節で述べる.

4.2 MOP1 : 拍検出の正確性

測定方法: メトロノームを 120 BPM に設定し, その拍に合わせて指揮者ノードを 60 秒間 (120 拍) 振り, ファームウェアが検出した拍イベント (拍番号・デバイス時刻・推定 BPM) をシリアル出力から記録した. 判定基準は検出率 (検出拍数/実施拍数) 90%以上である.

結果: 図 8 に検出された拍間隔の時系列を示す. 120 回の振りに対して検出は 120 拍 (検出率 100.0%) であり, 取りこぼしも二重検出もなかった. 拍間隔は全区間で理論値 500 ms の周りに分布し, 平均 501.2 ms・標準偏差 44.4 ms (変動係数 8.9%) であった. 拍間隔から算出した検出テンポは 119.7 BPM で, メトロノームの設定値 120 BPM との差は 0.3 BPM である. 図中で拍番号 1~40 の区間は間隔の振れ幅が約 370~620 ms と大きく, 拍番号 70 以降は 500 ms 付近に収束している. これは検出誤差ではなく振り手の慣れによるばらつきの減少であり, 人間の振り

の揺らぎを含んだ状態でも取りこぼしなく 1 振り 1 拍が成立することを示している。データは results/mop1/20260702_205848.csv である。

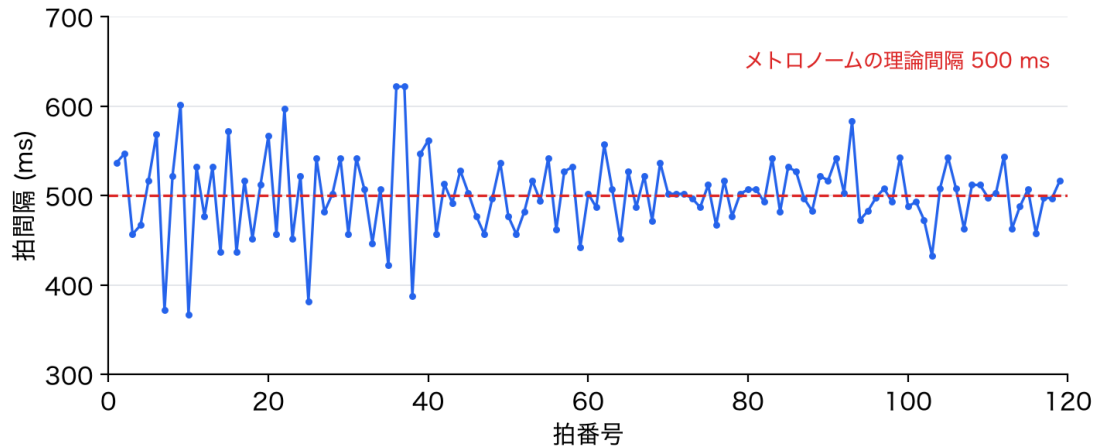


図 8 拍検出の拍間隔の時系列 (メトロノーム 120 BPM・120 拍)

4.3 MOP2：音階の誤差

測定方法：計画書では「合成出力音を録音し周波数分析して平均律理論音高と比較する」と定義していたが、実音の録音にはサウンドカードの D/A 特性や部屋の音響が混入し、測りたい対象である「楽器定義 JSON と合成方法に由来する音階誤差」だけを分離できない。そこで Processing の加算合成アルゴリズムを Python へ忠実に移植し、「現在の楽器定義と発音方法で鳴るはずの波形」を数値合成して周波数解析する方法に変更した (表 5)。検出される誤差が 100%音源データと合成式に由来することが保証され、再現も決定論的になる。楽譜で使用する 6 音 (C・D・E・F・G・A) を音高を持つ 4 楽器それぞれの実発音オクターブで合成し (全 24 音)、瞬時周波数法と FFT ピーク法の独立 2 手法で基音周波数を推定して平均律理論値との cent 差を求めた。推定器自体は既知信号による 10 項目のセルフテスト (純音・デチューン・ビブラート・トレモロ) で誤差 0.02 cent 未満を確認しており、測定対象の許容値 3.6 cent に対して十分な分解能を持つ。なお許容値は、カラオケ採点用の高分解能ピッチ抽出法の研究 [11] で報告された抽出精度 (3.6 cent 未満) に基づいてチームで設定した値である。

結果：図 9 に楽器別・音名別の音階誤差を示す。全 24 音の平均絶対誤差は 0.737 cent, 最大でも 1.907 cent であり、許容値 3.6 cent を最大値で判定しても満たす。図から読み取れるとおり、誤差は音名によらず楽器ごとにほぼ一定の系統オフセットであり、トランペットが -0.86 cent, ホルンが -1.91 cent, フルートが +0.10 cent, オルガンが +0.08 cent である。この誤差の由来は 5.4 節で分析する。2 手法の推定値の差は最大 0.1 cent 程度で一致した。データは results/mop2/ (計測 CSV と評価記録 evaluation.md), 検証スクリプトは tools/verification/scripts/mop2_pitch_error.py である。

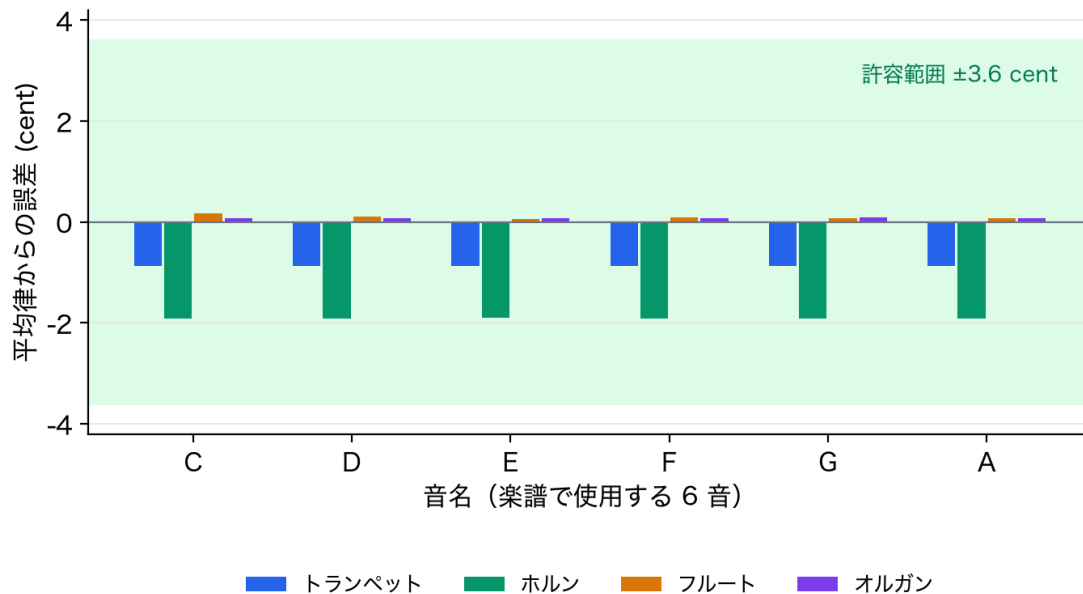


図9 音階の誤差（楽器別・音名別の平均律からのずれ．音高を持つ4楽器・24音）

4.4 MOP3：楽譜との相違

測定方法：演奏中に各楽器ノードが送出した NOTE のノート番号列をログに記録し，ファームウェアに埋め込んだ楽譜データ（score_data.cpp）の期待値列と拍番号ごとに突き合わせ，不一致（誤ノート）を数えた．判定基準は誤ノート0件である．

結果：全5パートで誤ノートは0件であった（2026年7月9日測定．データは results/mop3/配下の 20260709_235548_summary.txt）．なお開発中期（7月2日）の測定では node_02 に24件の誤ノートが出たことがあるが，これは後述する WiFi パケットロス（4.10 節）由来であり，ノードの物理配置の変更でパケットロスが解消すると同時に誤ノートも0件になった．誤ノートの原因が楽譜ロジックではなくパケット欠落であったことは，この2つの指標が同時に改善したことから裏付けられる．

4.5 MOP4：楽器間同期誤差

測定方法：全5楽器に計測ファーム（MOP_TEST=4）を書き込み，発音発火の瞬間の推定マスタ時刻をデバイス側で記録した．同一拍番号について5ノードの発火時刻の最大値と最小値の差（最速ノードと最遅ノードの発音時刻差）を同期誤差と定義する．USB シリアルへの到着ジッタが混入しないデバイス側計測である．判定基準は20ms以内である．最終構成（2026年7月11日，173拍×5ノード）の結果を主として報告し，対策前後の3回の実測の比較は5.2節で述べる．データは results/mop4/20260711_154006.csv（生ログは logs/test_20260711_153711.log）である．

結果：図10に同期誤差の時系列を，図11にその分布を示す．173拍の中央値は7ms・平均10.8ms（標準偏差12.6ms）であり，90.8%の拍が目標の20ms以内に収まった．中央値は目標値の

約 3 分の 1 であり、数十 ms のずれは主観的に同時とみなされるという知見 [4] に照らせば、合奏の同時性は聴感上成立している。実際、成果発表会の実演でもチーム内外からタイミングのずれの指摘はなかった。またノード別の平均偏差は $-1.7 \sim +3.1$ ms の範囲で、系統的に先行・遅延する楽器は存在しなかった。

一方で図 10 から読み取れるとおり、時系列にはおよそ 10 拍に 1 回弱の頻度で 30~65 ms の単発スパイクが現れ、最大値は 65 ms に達した。このため「全拍で 20 ms 以内」という判定基準そのものは満たせず、超過は 16 拍 (9.2%) である。図 11 の分布でも、本体は 0~20 ms に集中する（この範囲だけ見ればほぼ左右対称で中央値 7 ms）一方、30 ms 以上に薄い裾が延びており、95 パーセンタイルは 48.6 ms である。このスパイクの原因は楽器マイコンの無線モジュールに由来する周期的な処理停止であることを特定しており、5.3 節で分析する。

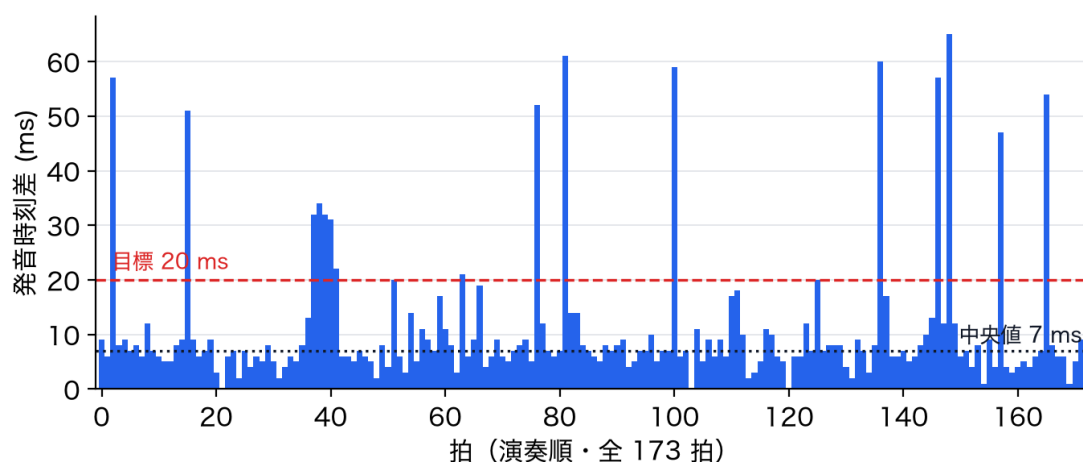


図 10 楽器間同期誤差の時系列（各拍の最速ノードと最遅ノードの発音時刻差、173 拍）

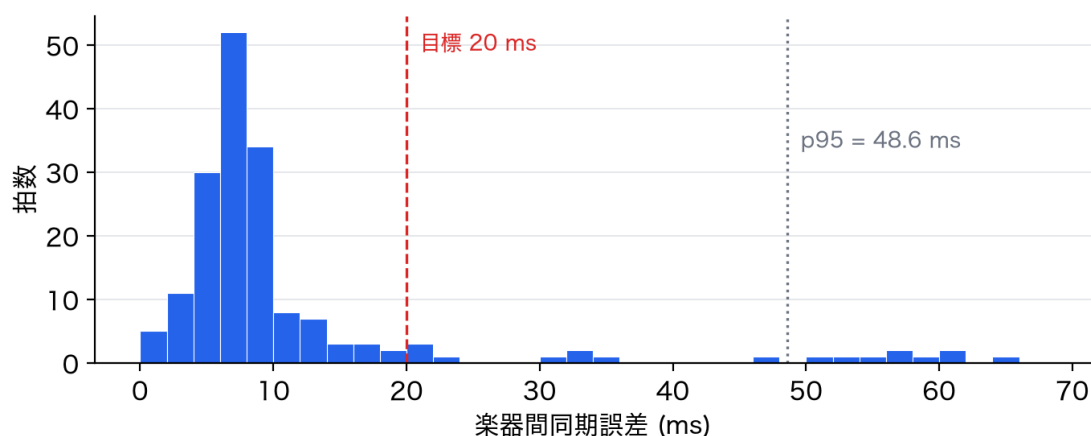


図 11 楽器間同期誤差の分布（図 10 と同一データのヒストグラム）

4.6 MOP5：指揮から楽器への通信遅延

測定方法：計画書の MOP5 は「指揮から楽器への通信遅延 30ms 以内」である。BEAT パケットの送信時マスタ時刻（ヘッダの `timestampMs`）と楽器側の受信時刻の差から配送遅延を求める。ただし指揮者から楽器への片方向通信では両者の時計オフセットが未知であり、絶対的な片道遅延は原理的に直接測定できない。そこでノードごとに観測値 s_i （式 (5)）の時系列を一次回帰してクロックスキューを補正し、残差の上側包絡（上位 2% 点）を「窓内で最も速く届いたパケット＝遅延最小」の基準線とみなして、そこからの下方距離を配送遅延の推定値とした。この値は真の遅延から共通の定数を引いた相対値（真の遅延の下限側の推定）であり、遅延のばらつきと分布の形は正しく捉えられる。測定は最終構成（2026 年 7 月 11 日）の 865 受信（173 拍 × 5 ノード）で行った。データは `results/mop5/20260711_154006.csv`、集計は付属スクリプト（`fig/make_report_graphs.py`）である。

結果：図 12 に配送遅延の分布を示す。遅延は 0～215 ms のほぼ全域に広がる一様分布状であり、平均 100 ms・中央値 100 ms・95 パーセンタイル 197 ms・最大 215 ms であった。目標の 30 ms 以内に収まった受信は全体の 14.2% にすぎず、**MOP5 は未達である**。図から読み取れる重要な特徴は、分布が特定の値に集中せず 0 ms から約 205 ms まで平坦に延びていることで、これは遅延が「電波状況によるゆらぎ」ではなく「周期的な配送タイミングまでの待ち時間」であることを示唆する。実際、この分布はアクセスポイントのマルチキャスト配送が約 204.8 ms 周期のバースト状になっていることで完全に説明でき、ビーコン設定の変更を含む 3 回の実測すべてで同じ周期が再現した。原因の特定と対策、およびこの指標そのものの設計上の問題は 5.1 節・5.2 節で詳述する。

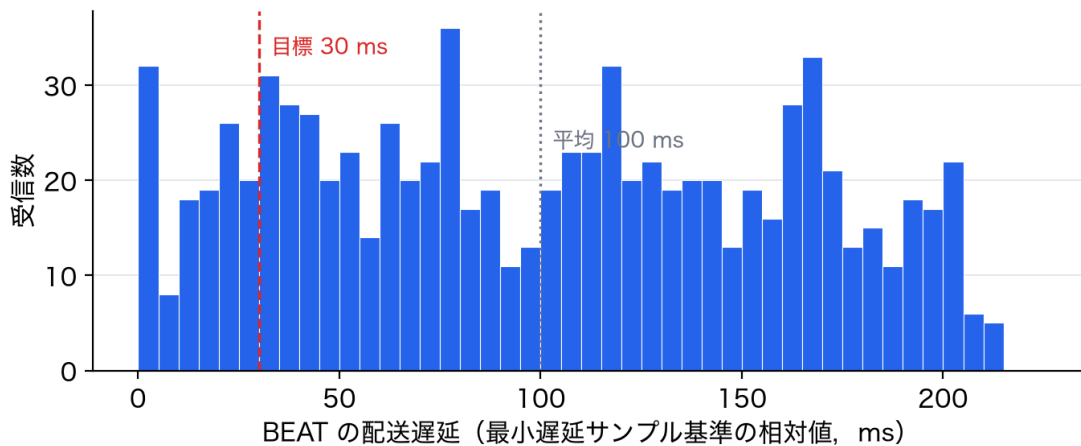


図 12 BEAT の配送遅延の分布（最終構成，865 受信．最小遅延サンプル基準の相対値）

4.7 MOP6：テンポ追従の遅延

測定方法：指揮の振りを約 80 BPM から約 130 BPM へ切り替え、楽器ノードが受信するテンポ（CTRL の BPM 値）が新テンポの $\pm 15\%$ 以内に入るまでの遅れを拍数に換算した。指揮者側の

BPM が拍間で 10%以上変化した時点をもテンポ変化の起点と定義する。判定基準は 2 拍以内である。データは results/mop6/20260702_212701.csv である。

結果：図 13 に楽器 4 台が受信したテンポの時系列を示す (node_02 はこの計測時に WiFi 接続品質の問題でログが取得できず、後日の配置変更で解消した。4.10 節)。経過 29.1 s の時点でテンポ変化が検出され、各ノードの受信 BPM は約 80 BPM から 3 秒程度かけて約 130 BPM へ滑らかに遷移している。これは式 (4) の EMA 平滑化 ($\beta = 0.30$) による意図した挙動である。追従遅延は node_04 の 1.2 拍が最大で、node_03 は 0.2 拍、node_05・node_06 は 0.0 拍であり、全ノードが目標の 2 拍以内に収まった。また図の全区間で 4 本の曲線がほぼ重なっていることから、テンポ情報が全楽器へ一様に行き渡っていることも読み取れる。

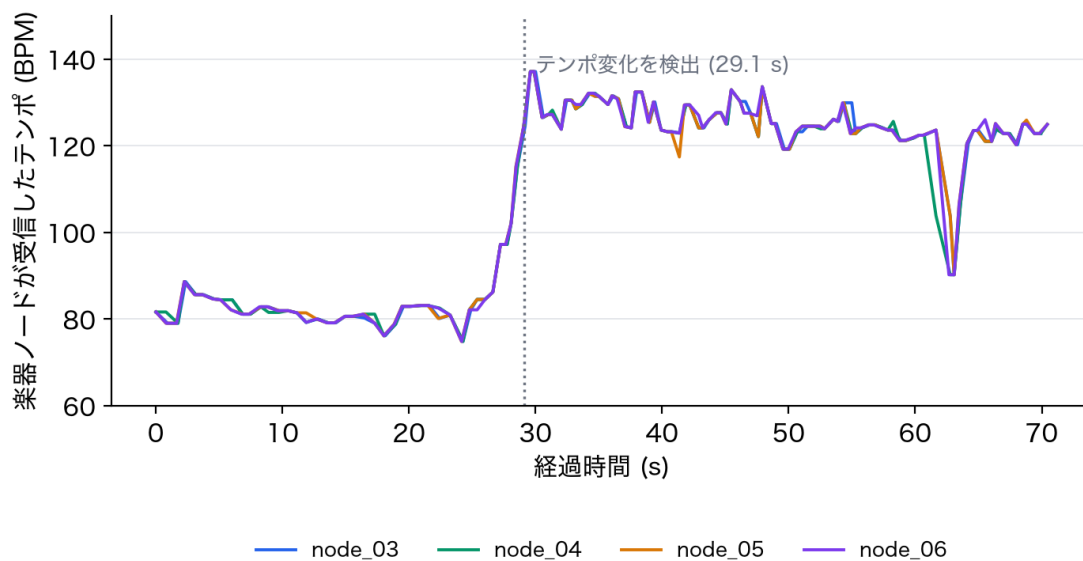


図 13 テンポ追従の時系列 (約 80 BPM → 約 130 BPM の切替, 楽器 4 台)

4.8 MOP7：起動時間

測定方法：各ノードの電源投入 (リセット) から演奏可能状態までの時間をデバイス内蔵の millis() で記録した。演奏可能状態は、指揮者はキャリブレーション完了 (振り待ち受け開始)、楽器は WiFi 接続と時刻同期の完了と定義する。判定基準は 5 秒以内である。データは results/mop7/20260702_205051.csv である。

結果：図 14 にノード別の起動時間を示す。最長は指揮者ノードの 2.33 s (2 s 間の静止キャリブレーションを含む)、楽器ノードは 0.93 ~ 1.48 s であり、全ノードが目標の 5 s 以内に収まった。なお計測ファームはシリアル初期化の待ち時間 (最大 1.5 s) を含むため、この値は本番構成より大きめの保守的な値である。

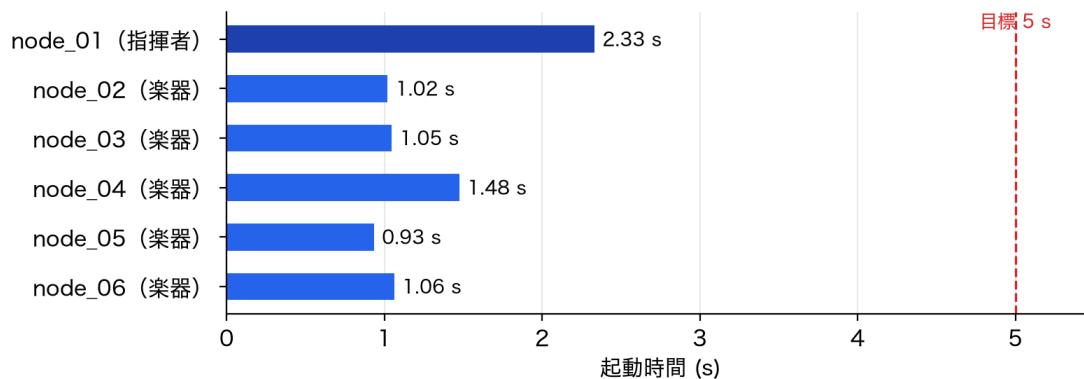


図 14 ノード別の起動時間（電源投入から演奏可能状態まで）

4.9 MOP8：CPU 負荷

測定方法：指揮者ノードのメインループ3フェーズ（入力・ロジック・出力）の実行時間を `micros()` で毎周期計測し、12,326 サンプルを収集した（集計時は初期化コストの影響を除くため先頭 20 サンプルを除外）。判定基準は、IMU のサンプリング周期 5 ms の中で最も重い入力フェーズが 2 ms 以内であることとした。データは `results/mop8/20260702_210538.csv` である。

結果：図 15 に入力フェーズ処理時間の分布を示す。平均 $71\mu\text{s}$ ・99 パーセンタイル $620\mu\text{s}$ ・最大 $724\mu\text{s}$ であり、目標 $2000\mu\text{s}$ の 4 割にも達しない。分布は約 $40\mu\text{s}$ （IMU 読取りのみの周期）と約 $500\sim 690\mu\text{s}$ （WiFi 送信処理を含む周期）の二峰性を示すが、最悪ケースでも判定基準に対して 2.8 倍の余裕がある。3 フェーズ合計でも最大 1.50 ms であり、楽器ノードの制御周期 2 ms 化（3.3.2 節）を支える根拠となった。

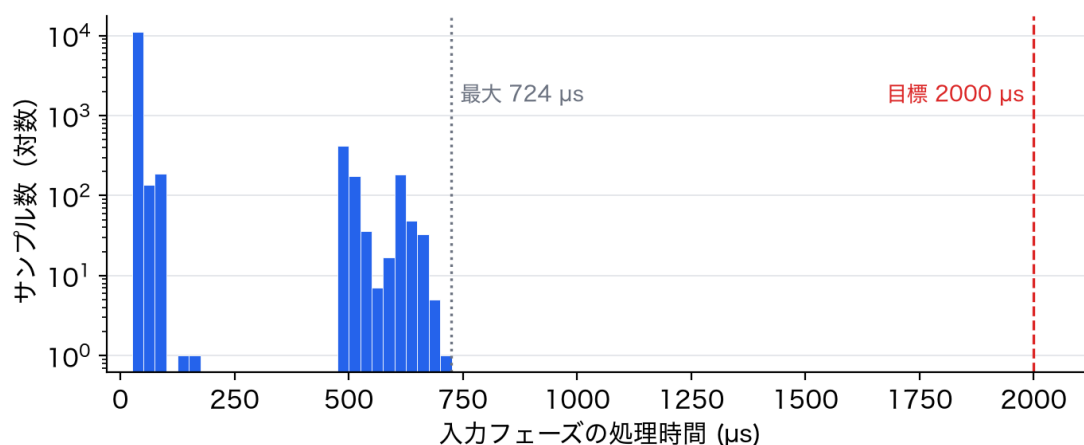


図 15 入力フェーズの処理時間分布（12,326 サンプル・縦軸は対数）

4.10 MOP9：パケットロス耐性

測定方法：演奏中に各楽器ノードが受信した BEAT の拍番号列を記録し，期待拍数に対する欠落数と最大連続欠落を求めた．判定基準は欠落 5%以下である．データは results/mop9/20260709_233828_summary.txt である．

結果：表 9 に示すとおり，全 5 ノードで欠落 0 (0.0%)・最大連続欠落 0 拍であった．なお開発中期（7 月 2 日）の測定では node_02 のみ 13.3%の欠落が出たことがあり，ノード間の距離を確保する物理配置の変更で解消した．同一パケット 4 連送（2.5 節）の効果と合わせ，発表会構成では取りこぼしのない配信が成立している．

表 9 パケットロス測定の結果（2026 年 7 月 9 日）

ノード	期待拍数	受信拍数	欠落	最大連続欠落
node_02	143	143	0 (0.0%)	0 拍
node_03	143	143	0 (0.0%)	0 拍
node_04	141	141	0 (0.0%)	0 拍
node_05	141	141	0 (0.0%)	0 拍
node_06	132	132	0 (0.0%)	0 拍

4.11 成果発表会におけるエンタテインメント性評価

方法：成果発表会（2026 年 7 月 8 日）で本システムの実演を行い，参加者にアンケートを実施した．評価項目は「没入感・臨場感」「一体感・同期の心地よさ」「演出の華やかさ」「楽しさ・高揚感」の 4 項目（各 7 段階，1：低い～7：高い）と自由記述であり，チーム 23 への回答は 43 件であった．データは work/shiozawa/最終報告書/data/entertainment_survey_team23.csv である．

結果：表 10 に集計値を，図 16 に回答分布を示す．「楽しさ・高揚感」が平均 6.21（中央値 7）で最も高く，43 件中 22 件が最高評価の 7 を付けた．「没入感・臨場感」は平均 6.09（中央値 6），「一体感・同期の心地よさ」は平均 6.07（中央値 6）で，いずれも回答の過半が 6 以上に集中している．一方「演出の華やかさ」は平均 5.14（中央値 5）で他の 3 項目より約 1 段低く，図 16 のとおり分布の山も 4～5 に位置する．

表 10 エンタテインメント性評価の集計（7 段階評価・43 件）

評価項目	平均	中央値	標準偏差
没入感・臨場感	6.09	6	0.97
一体感・同期の心地よさ	6.07	6	0.88
演出の華やかさ	5.14	5	1.26
楽しさ・高揚感	6.21	7	1.04

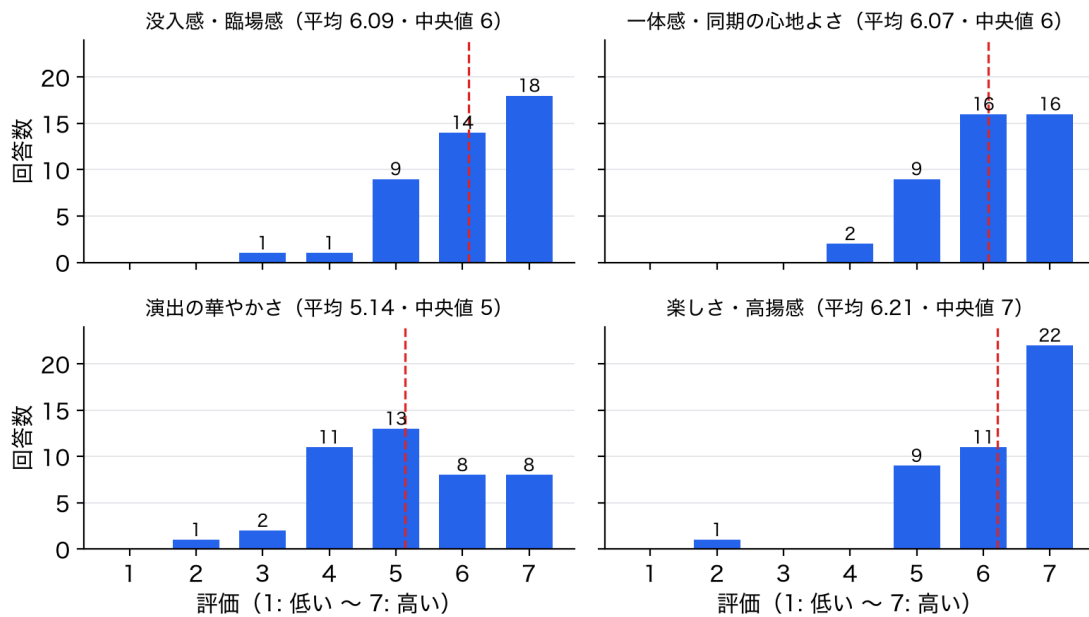


図 16 エンタテインメント性評価の回答分布 (各項目 $n = 43$ 。破線は平均値)

自由記述は 6 件あり、「自分の動きでテンポなどを決められるので、没入感がダントツで高いなと感じました。」「指揮者を体験できる楽しさがあった」「自分も指揮者の棒を振ることが出来てとても楽しかった。」「bpm 変更を指揮棒にしたことがよかった」「指揮者できる」など、6 件中 5 件が指揮者体験そのものへの言及であった。結果の解釈は 5.5 節で述べる。

5 考察

5.1 MOP5 未達の原因：マルチキャストのバースト配送

4.6 節で示したとおり、指揮から楽器への通信遅延は平均 100 ms と目標 30 ms を大幅に超過した。原因を生ログの時系列解析で追跡した結果、SoftAP の UDP マルチキャストが約 204.8 ms 周期のバースト状に配送されていることを特定した。図 17 に同一ノードにおける BEAT の受信間隔の分布を示す。拍は約 500 ms 間隔で送信されているにもかかわらず、受信間隔は 205・410・615 ms というとびとびの値、すなわち 204.8 ms の整数倍の格子上に集中しており（格子から ± 15 ms 以内が 95.8%）、パケットが連続的にではなく「かたまり」でしか届いていないことが読み取れる。

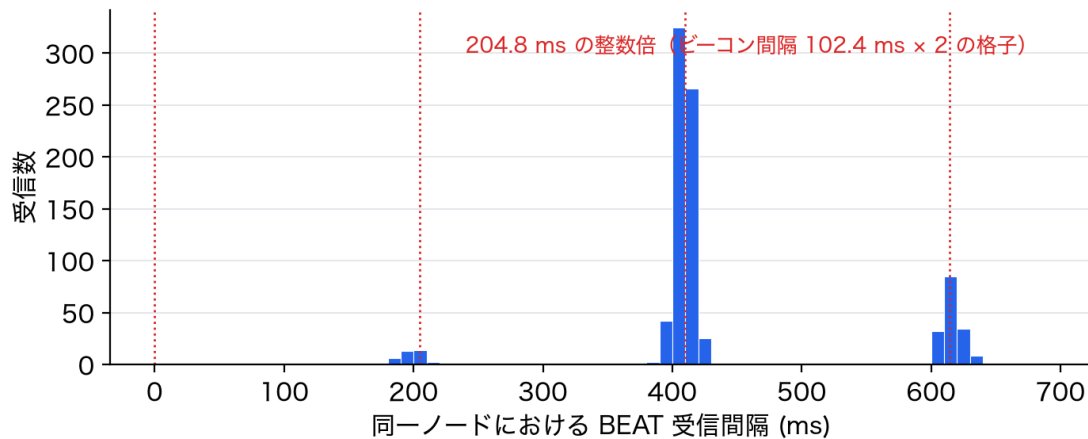


図 17 同一ノードにおける BEAT 受信間隔の分布（受信が 204.8 ms 周期の格子に乗る）

これは IEEE 802.11 の省電力機構 [12] に由来する挙動である．アクセスポイントは省電力モードの端末が 1 台でも接続していると，マルチキャストフレームを即時送信せず，DTIM (Delivery Traffic Indication Message) ビーコンのタイミングまでバッファリングして一括送出する．本システムのビーコン間隔は 102.4 ms (100 TU) であり，実測周期 204.8 ms はそのちょうど 2 倍にあたる．楽器側の UNO R4 WiFi の無線部 (ESP32-S3 モジュール) は省電力が既定で有効であり，ライブラリに省電力を制御する API も存在しないため，この配送特性はホスト側ソフトウェアからは回避できない．拍は人間の指揮で打たれるためバースト周期に対して位相がランダムになり，配送遅延は「次の一括送出までの待ち時間」，すなわち区間 $[0, 204.8]$ ms のほぼ一様分布になる．これは図 12 の実測分布（平均 100 ms ・最大 215 ms）と一致する．

この結論は 3 つの独立した証拠で裏付けられる．第一に，バースト周期は 2026 年 7 月 10～11 日の 3 回の計測（ビーコン設定の変更を挟む）すべてで 204.0～205.1 ms に再現した．第二に，デバイス時計と PC 時計という独立の時計系で同じ周期が検出された．第三に，ビーコン間隔を半分 (100→50 TU) にする設定変更を試みたところ，設定自体は受理されたにもかかわらずバースト周期は 1 ms も変わらず，周期がビーコン設定ではなく DTIM 相当の機構で決まっていることが確認された．この試行は生成 AI の提案を実測で棄却した例として 3.5.2 節でも述べたとおりであり，効果のない設定は最終構成から撤去した．

以上から，MOP5 の目標「通信遅延 30 ms 以内」は本システムの構成（SoftAP + マルチキャスト + 省電力既定の無線モジュール）では物理的に達成不能であったと結論する．またこの分析は指標設計そのものへの反省も与える．本システムは発音予約方式 (2.4 節) を採るため，配送遅延は予約時刻までに届く限り演奏品質に影響しない．すなわち「生の通信遅延」はシステムのアーキテクチャが保証しようとする性質（発音の同時性）と対応しない下位層の物理量であり，指標としては「予約が成立したか」を測るべきであった．これは検証計画の段階で気づけたはずの設計ミスであり，指標はアーキテクチャが保証する性質に対応させて定義すべきであるという教訓を得た．

5.2 対策と効果：発音予約の再設計

前節の原因分析に基づき、遅延そのものは消せないという前提で、遅延の影響を打ち消す2つの対策を実装した。第一に発音予約の先読み時間をバースト周期に基づいて45→220ms（204.8ms＋余裕）へ拡大し、第二に時計オフセット推定をEMAからminフィルタ（式（6））へ変更した。EMAはバースト環境では観測値 s_i の平均的な位置、すなわち真値より約40～55ms遅れた推定値に収束し、発音時刻の遅れと遅刻量の過小計測（対策前の計測は真の遅刻を54～59ms過小表示していた）を招いていたためである。

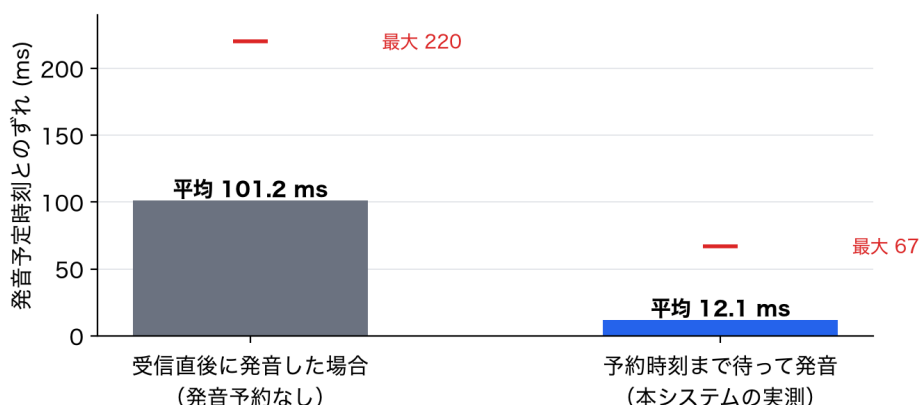


図 18 発音予約方式によるジッタ吸収の効果（発音予定時刻とのずれの比較、最終構成）

図 18 に発音予約方式の効果を示す。受信した瞬間に発音した場合（発音予約なし）の発音予定時刻とのずれは平均 101.2ms・最大 220ms に達するが、予約時刻まで待って発音する実測（本システム）では平均 12.1ms・中央値 6ms・最大 67ms まで抑えられる。生の配送遅延が平均 100ms という環境でも合奏がそろうのはこの機構によるものであり、「届くのはバラバラでも、鳴るのは同時」という設計意図が定量的に裏付けられた。

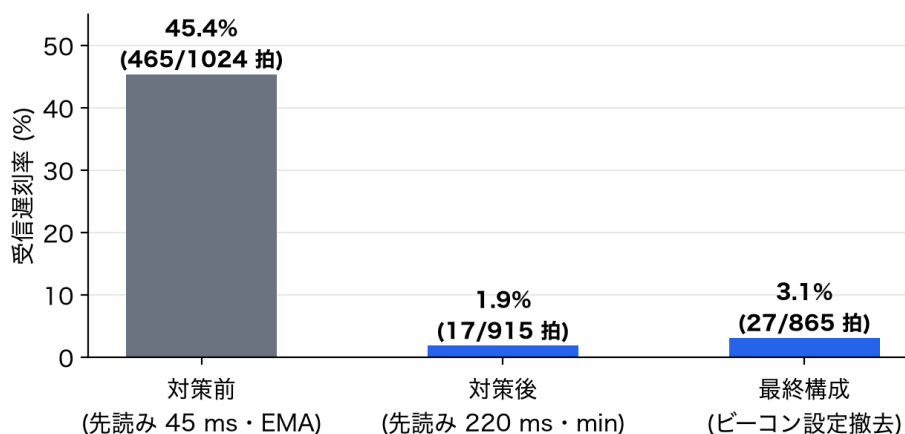


図 19 同期対策の前後での受信遅刻率（予約時刻に間に合わなかった拍の割合）の変化

表 11 同期対策の前後比較 (2026 年 7 月 10～11 日の 3 回の実測)

指標	対策前	対策後	最終構成
先読み時間	45 ms	220 ms	220 ms
受信遅刻率 (予約破綻率)	45.4 % (465/1024 拍)	1.9 % (17/915 拍)	3.1 % (27/865 拍)
受信遅刻の最大	98 ms	14 ms	33 ms
受信マージンの平均	+2.2 ms	+102.7 ms	+100.8 ms
発火遅刻 (中央値/最大)	8/101 ms	6/79 ms	6/67 ms
楽器間同期誤差 (中央値/最大)	8/32 ms	7/79 ms	7/65 ms

対策前後の変化を図 19 と表 11 に示す。予約時刻までに BEAT が届かなかった拍の割合 (受信遅刻率) は、対策前の 45.4 % から対策後 1.9 %・最終構成 3.1 % へと大幅に減少した。すなわち対策前は半数近くの拍で発音予約が機能しておらず「受信したら即発音」に近い状態だったものが、対策後はほぼ全拍で予約どおりの発音が成立している。遅刻した拍も捨てずに即時発音して回復するため演奏の欠落は生じず、遅刻量も最大 14～33 ms にとどまる。

この対策の代償として、指揮の振りから発音までの絶対遅延は約 220 ms + 発火遅刻 (平均約 232 ms, 120 BPM の約半拍) の固定遅延となった。対策前は平均約 125 ms と小さいが拍ごとに暴れる遅延であった。本システムでは「遅延のばらつきを固定遅延に置き換えて楽器間の同時性を確保する」トレードオフを選択した。楽器間がそろっている限り、一定の追従遅れは指揮行為の中で自然に吸収され、成果発表会の実演でも違和感の報告はなかった。

また min フィルタ化には計測の信頼性という副次的な成果もある。推定時計が最小遅延のサンプルに張り付くことで系統誤差がほぼ解消し、対策後の遅刻計測値は真の遅刻の下限として解釈可能な「正直な」値になった。性能の主張がどの時計基準に立つかを明確にできたことは、本検証全体の数値の信頼性を支えている。

5.3 MOP4 の残る誤差要因と判定基準の妥当性

4.5 節で示したとおり、楽器間同期誤差は中央値 7 ms と目標を大きく下回る一方、30～65 ms の単発スパイクが 9.2 % の拍で発生し、最大値判定では未達となった。原因を生ログの時系列解析で追跡した結果、楽器ノードのメインループがバースト到着から特定位相 (約 120～165 ms) の区間で約 40 ms 実行されない周期的なストールが全ノードで観測され、発音予定時刻がこの区間に落ちた拍だけが 30～60 ms 遅れて発火することを特定した。ストールはビーコン設定の変更前後で不変であったことから、UNO R4 WiFi の無線モジュール (ESP32-S3 ブリッジ) 内部の周期処理に由来する既存挙動と結論した。ホスト側ソフトウェアでの根治は難しく、根本対策には無線モジュール側の省電力設定の変更 (モジュールファームウェアの改造) が必要である。これは本システムの現構成における限界である。なお表 11 で対策後に最大値が 32→65～79 ms へ増えて見えるのは悪化ではなく、対策前は「受信即発音」に近くストール区間を踏む前に発音していたため、この既存挙動が露出していただけである (中央値と超過率は 3 回の実測で同水準)。

この経験から、判定基準の設計にも反省がある。ジッタを持つ通信系の品質を最大値で判定すると、中央値が目標の 3 分の 1 でも外れ値 1 拍で不合格になる。聴感品質に対応した分位点判定

(例：95 パーセンタイル $\leq$ しきい値，または超過率 $\leq N\%$) を計画段階で採用すべきであった。

5.4 MOP2 の測定方法の妥当性と誤差の由来

MOP2 は計画の「実音録音の周波数分析」から「合成アルゴリズムの忠実再現」へ測定方法を変更した (4.3 節)。この変更は測定対象の純化 (音源データと合成式に由来する誤差だけを取り出す) が目的であり、判定を有利にする変更ではない。実測された系統誤差の由来は楽器定義 JSON まで完全に追跡できた。合成音の音高は基音周波数に第 1 倍音の周波数比 `ratio` を乗じて決まるが、実楽器の録音から音色を抽出する際に宣言基音と実測第 1 部分音の位置がわずかに食い違い、トランペットで 0.9995 (−0.87 cent 相当)、ホルンで 0.9989 (−1.91 cent 相当) という 1.0 でない値が残っていた。これが図 9 の音名によらない一定オフセットの正体である。非調和性パラメータの寄与は +0.04~+0.08 cent で無視できる。なお本測定は合成波形のシミュレーションであり、実際の D/A 変換以降で加わる誤差は含まないが、それらは通常 cent 未満であり結論を変えない。必要であれば `ratio` を 1.0 に正規化することで誤差はさらに縮小できるが、現状でも許容値の 2 割程度であり音楽的には知覚不能なため対応不要と判断した。

5.5 エンタテインメント性評価の考察

4.11 節のアンケート結果は、本システムの設計意図と整合的に解釈できる。最も高い「楽しさ・高揚感」(平均 6.21) と「没入感・臨場感」(平均 6.09) は、自由記述の 6 件中 5 件が「指揮者を体験できる」ことへの言及であったことと合わせると、「自分の腕の振り一つで 5 台の楽器が動く」という合奏の構図の体験そのものが評価されたと考えられる。これは 1 章で述べた、一人一音の変換にとどまらないシステムの狙いが聴衆に伝わったことを示す。「一体感・同期の心地よさ」(平均 6.07) も 6 以上に回答が集中しており、4.5 節の同期性能 (中央値 7 ms) が主観評価としても成立していたことを裏付ける。

一方「演出の華やかさ」(平均 5.14) は他項目より約 1 段低い。本システムの出力は音と各ノードの LED 表示のみで、視覚的な演出 (照明・映像・楽器の動き) を持たないことが素直に反映されたと考えられる。音の同期という技術的な核が高評価である以上、改善の方向は音楽と同期した視覚演出 (拍に同期した照明、PC 画面の演出強化) の追加であり、既に全ノードが共有している拍・テンポ情報をそのまま駆動源にできる。

なお本評価の限界として、回答者は成果発表会の参加者 (同じ授業を受講する学生が中心) であり、中立的な第三者による評価ではないこと、および他チームとの相対比較を目的とした設問設計ではないことを付記する。

5.6 システムの限界と改善案

以上の分析を踏まえ、本システムの現構成における限界と改善案を整理する。第一に、楽器間同期誤差の裾 (最大 65 ms) の原因である無線モジュール内部の周期ストールは、ホスト側からは回避できない。改善には無線モジュールの省電力設定の変更 (モジュールファームウェアの改造) の検証が必要であり、ストールが解消すれば発火遅刻の 95 パーセンタイルは約 9 ms まで下がると試

算している。ループ間隔のハートビート計測機構は整備済みであり、ストールの正確な粒度の特定から着手できる。第二に、指揮から発音までの固定遅延約 220 ms は、テンポの急変に対する応答を約半拍遅らせる。遅延と同時性は本構成ではトレードオフの関係にあり、遅延も縮めたい場合はマルチキャストをやめユニキャスト 5 本に切り替える（省電力バッファリングを回避する）比較実験が改善候補である。第三に、評価指標の設計として、ジッタを持つ系の品質は分位点で判定する基準を計画段階から採用すべきである。第四に、エンタテインメント面では拍同期の視覚演出の追加が有効である。

6 おわりに

本プロジェクトでは、指揮者の IMU ジェスチャで 5 台の楽器ノードと PC 音色合成を駆動する分散合奏システム「タクトーン」を製作した。報告者は Arduino 系プログラム全般（指揮者・楽器・共通層）と性能検証を担当し、次の成果を得た。

第一に、経路長積分に基づくゲート式の拍検出を設計・実装し、メトロノーム 120 拍に対して検出率 100.0% を達成した。第二に、発音予約と時計同期による同期機構を設計・実装し、楽器間の発音時刻差を中央値 7 ms（目標 20 ms の約 3 分の 1）にそろえた。9 項目の性能指標は 7 項目で目標を達成した。第三に、未達となった 2 項目については原因を実測で特定した。指揮から楽器への通信遅延（MOP5、実測平均 100 ms）はアクセスポイントの省電力機構に由来する約 205 ms 周期のバースト配送が原因で、本構成では目標 30 ms が達成不能であること、楽器間同期誤差（MOP4）の最大値超過は無線モジュール内部の周期ストールが原因であることを明らかにした。そのうえで、遅延そのものではなく遅延の影響を打ち消す対策（先読み時間の再設計 45→220 ms・時計同期の min フィルタ化）により、発音予約が破綻する拍を 45.4% から 3.1% へ減らし、「届くのはバラバラでも、鳴るのは同時」という設計原理が成立することを延べ 560 拍 × 5 ノードの実測で示した。第四に、成果発表会の聴衆評価（43 件）で「楽しさ・高揚感」平均 6.21（7 段階）を得て、指揮者体験というシステムの狙いが聴衆に伝わることを確認した。

序論で掲げた「無線ジッタの下で複数楽器の発音をそろえる」という技術課題に対しては、楽器間同期の観点では目標を上回る水準を達成した一方、通信遅延という指標の観点では未達であり、その原因が指標設計（下位層の物理量を目標化したこと）と環境の実挙動（バースト配送）の両方にあることを数値で示した。指標はアーキテクチャが保証する性質に対応させて定義すべきであるという教訓は、本プロジェクトを超えて通用する知見である。

残された課題は、無線モジュール内部の周期ストールの根本解決、固定遅延 220 ms と即応性のトレードオフの検討、および分位点ベースの品質判定基準の整備である。本システムは、標準的な無線 LAN と安価なマイコンのみで、未経験者が指揮者として合奏を駆動する体験が実用水準で成立することを示した。

参考文献

- [1] 川合雄佑, 伊藤彰教, 「非接触型電子楽器の開発を通じたサウンドデザイン」, 先端芸術音楽創作学会会報, Vol. 14, No. 1, pp. 14–19, 2022.
- [2] 任天堂株式会社, 「Wii Music 公式サイト」, <https://www.nintendo.co.jp/wii/r64j/index.html>, 2008 (2026-07-14 閲覧) .
- [3] 久松剛, 「分散環境におけるフィードバックを用いたオーケストラ演奏機構の構築」, 慶應義塾大学 環境情報学部 卒業論文, 2003.
- [4] 河瀬諭, 「合奏における演奏者間コミュニケーション—タイミング調整とその手がかり—」, 心理学評論, Vol. 57, No. 4, pp. 495–510, 2014.
- [5] Arduino, “UNO R4 WiFi Documentation”, <https://docs.arduino.cc/hardware/uno-r4-wifi/> (2026-07-14 閲覧) .
- [6] Processing Foundation, “Processing Reference”, <https://processing.org/reference/> (2026-07-14 閲覧) .
- [7] D. Mills, J. Martin, J. Burbank, W. Kasch, “Network Time Protocol Version 4: Protocol and Algorithms Specification”, RFC 5905, IETF, 2010.
- [8] 中村栄太, 武田晴登, 山本龍一, 齋藤康之, 酒向慎司, 嵯峨山茂樹, 「任意箇所への弾き直し・弾き飛ばしを含む演奏に追従可能な楽譜追跡と自動伴奏」, 情報処理学会論文誌, Vol. 54, No. 4, pp. 1338–1349, 2013.
- [9] 塩澤匠生, 「Embedded-Module-Architecture」, <https://github.com/takushio2525/Embedded-Module-Architecture> (2026-07-14 閲覧) .
- [10] PlatformIO Labs, “PlatformIO Documentation”, <https://docs.platformio.org/en/latest/> (2026-07-14 閲覧) .
- [11] 竹内英世, 梅崎太造, 保黒政大, 「カラオケ採点用の高分解能ピッチ抽出法」, 電気学会論文誌 C, Vol. 129, No. 10, pp. 1889–1901, 2009.
- [12] IEEE, “IEEE Standard for Information Technology—Telecommunications and Information Exchange between Systems—Local and Metropolitan Area Networks—Specific Requirements—Part 11: Wireless LAN Medium Access Control (MAC) and Physical Layer (PHY) Specifications”, IEEE Std 802.11-2020, 2021.

A プログラムソース

報告者が担当したファームウェアのうち、本文で説明した中核部分のソースコードを掲載する。リスト 3 は全ノード共通の通信パケット定義 (2.5 節), リスト 4 は楽器ノードの受信処理と時計同期 min フィルタ (2.4 節), リスト 5 は楽器ノードの発音予約発火と輪唱進行ロジック (2.6 節), リ

スト 6 は指揮者ノードの拍検出・テンポ推定・状態遷移ロジック（2.3 節）である。ファームウェア全体（7 ノード分の全ソース・ビルド設定・検証スクリプトを含む）はチームの GitHub リポジトリ <https://github.com/takushio2525/2026-hackathon-team23> で管理している。なお組版の制約により、コメント中の記号「×」と「§」はそれぞれ「x」と「Sec.」に置換して掲載している。

リスト 3 通信パケット定義 (common/lib/OrcProtocol/OrcProtocol.h)

```

1 // Build (run from project root) — shared by node_01~node_05 of production:
2 //   pio run -d firmware/production/node_01      # 指揮者ノード
3 //   pio run -d firmware/production/node_02      # 輪唱 声部 1
4 //
5 // CTRL / BEAT / NOTE / UI 共通の 20 バイト固定パケット定義
6 // 仕様: 共通ヘッダ 12 B + ペイロード 8 B、リトルエンディアン
7 //
8 // test_v2 の変更点: NotePayload の旧 reserved[0] を instrumentId に充てた。
9 // 楽器ノードは「輪唱の声部 = 楽器番号」を固定で持ち、PC 側 (
10 //   orchestra_resynth) は
11 // この番号で読み込み済みの楽器定義 (sound_lab の JSON) を選んで加算合成す
12 // る。
13 //
14 // production の変更点 (ゲームモード):
15 //   1. CtrlPayload の旧 reserved[4] を mode/navCursor/targetBpm/score に
16 //     フィールド化。
17 //     自由演奏/ゲームのモード・メニューカーソル・目標テンポ・得点を指揮者→
18 //     楽器へ載せる。
19 //     「画面」は (state, mode) から PC が導出するのでバイトは消費しない。
20 //   2. PKT_UI (type=4) を追加。楽器ノード→PC の USB シリアル専用で UI 状態
21 //     を中継する。
22 //     UDP マルチキャストには一切流さない (同期経路 CTRL/BEAT/NOTE に無影響)
23 //     。
24 #pragma once
25 #include <stdint.h>
26 #include <stddef.h>
27
28 namespace orc {
29
30 // ASCII "OR" (リトルエンディアンで 0x52 0x4F)
31 constexpr uint16_t MAGIC = 0x4F52;
32 constexpr uint8_t  PROTOCOL_VERSION = 0x01;
33
34 enum PacketType : uint8_t {
35     PKT_CTRL = 1,
36     PKT_BEAT = 2,
37     PKT_NOTE = 3,
38     PKT_UI   = 4,    // production: 楽器→PC の UI 状態中継 (USB シリアル専用)
39 };
40
41 constexpr size_t HEADER_SIZE = 12;

```



```

36 constexpr size_t PACKET_SIZE = 20;
37
38 #pragma pack(push, 1)
39 struct PacketHeader {
40     uint16_t magic;           // 0x4F52
41     uint8_t  version;        // 0x01
42     uint8_t  type;           // PacketType
43     uint32_t seq;            // 単調増加
44     uint32_t timestampMs;    // 送信時のマスタ時刻
45 };
46
47 struct CtrlPayload {
48     uint16_t bpmQ8;          // BPM x 8 (例: 120.5 → 964)。実振り BPM (自由演奏/ゲーム共通)
49     uint8_t  velocity;       // 0-127
50     uint8_t  state;          // 0=Idle 1=Calibrating 2=Conducting 3=Fallback 4=Menu 5=Result
51     // — production ゲームモード: 旧 reserved[4] をフィールド化 —
52     // 画面は (state, mode) から PC が導出するのでここには持たない。カーソルだけ別バイト。
53     uint8_t  mode;           // 0=自由演奏 / 1=ゲーム
54     uint8_t  navCursor;      // メニューカーソル位置 0..N (Menu/Result で有効)
55     uint8_t  targetBpm;      // ゲーム目標テンポ (生 BPM 40-240, 0=未設定/自由演奏では無視)
56     uint8_t  score;          // ゲーム得点 0-100, 0xFF=未確定 (Menu/Result で有効)
57 };
58
59 struct BeatPayload {
60     uint16_t beatNo;
61     uint8_t  reserved[2];
62     uint32_t playAtMasterMs; // マスタ時刻でこの ms に発音せよ
63 };
64
65 struct NotePayload {
66     uint8_t  partId;          // test_v2 は 0x02-0x04 / production は 0x02-0x05
67     // production 想定は 0x02-0x06 (ADR-0004 改訂版で楽器 5 台 = 金管 4 + ドラム 1)
68     uint8_t  noteNumber;      // MIDI 0-127, 60=C4 (高さ)
69     uint8_t  velocity;        // 0-127
70     uint8_t  gate;            // 1=NoteOn, 0=NoteOff
71     uint16_t durationMs;      // 発音予定長 (長さ)
72     uint8_t  instrumentId;    // 0..N-1: PC 側で読み込んだ楽器定義のインデックス (楽器番号)
73     uint8_t  reserved;        // 0 埋め
74 };
75 // production: 楽器ノード → PC への UI 状態中継ペイロード (USB シリアル専用)

```

```

76 // node_02 が受信 CTRL の中身を低頻度 (変化時 + 最大 5Hz) で PC へ転送し、
   Processing が
77 // (state, mode) から画面を自動判定する。UDP には流れない。
78 struct UiPayload {
79     uint8_t state;          // 0..5 (CtrlPayload.state と同義: Idle/
   Calibrating/Conducting/Fallback/Menu/Result)
80     uint8_t mode;          // 0=自由演奏 / 1=ゲーム
81     uint8_t navCursor;     // メニューカーソル位置 (Menu/Result で有効)
82     uint8_t targetBpm;     // ゲーム目標テンポ (生 BPM)
83     uint8_t score;         // ゲーム得点 0-100 / 0xFF=未確定 (Menu/Result で
   有効)
84     uint8_t partId;        // 中継元の楽器ノード ID (PC の役割判定用: 0x02=メ
   イン操作 UI)
85     uint16_t bpmQ8;        // 実振り BPM x8 (演奏画面のテンポ表示用)
86 };
87
88 struct CtrlPacket {
89     PacketHeader header;
90     CtrlPayload payload;
91 };
92
93 struct BeatPacket {
94     PacketHeader header;
95     BeatPayload payload;
96 };
97
98 struct NotePacket {
99     PacketHeader header;
100     NotePayload payload;
101 };
102
103 struct UiPacket {
104     PacketHeader header;
105     UiPayload payload;
106 };
107 #pragma pack(pop)
108
109 static_assert(sizeof(PacketHeader) == HEADER_SIZE, "header_must_be_12_B");
110 static_assert(sizeof(CtrlPacket) == PACKET_SIZE, "ctrl_packet_must_be_20_B");
111 static_assert(sizeof(BeatPacket) == PACKET_SIZE, "beat_packet_must_be_20_B");
112 static_assert(sizeof(NotePacket) == PACKET_SIZE, "note_packet_must_be_20_B");
113 static_assert(sizeof(UiPacket) == PACKET_SIZE, "ui_packet_must_be_20_B");
114
115 // magic / version の妥当性を確認しヘッダを取り出す
116 bool parseHeader(const uint8_t* buf, size_t len, PacketHeader& out);
117
118 } // namespace orc

```

リスト 4 楽器ノードの受信処理と時計同期 (common/lib/OrcReceiverModule/OrcReceiverModule.cpp)

```

1 // Build / Upload / Monitor (run from project root):
2 //   pio run -d firmware/production/node_XX
3 //   pio run -d firmware/production/node_XX -t upload
4 //   pio device monitor -d firmware/production/node_XX
5
6 #include "OrcReceiverModule.h"
7 #include "SystemData.h"
8 #include "MopTest.h"
9
10 namespace {
11
12 // マスターリセット (offset スナップ) 検知時の後始末。
13 // pending.playAtMasterMs は旧マスタ時計の値で、新 offset では発火判定が壊れ
14 // ため破棄。
15 // lastBeatNo は新しい連番 (1 から再開) と偶然一致して初拍を重複扱いで飲む事
16 // 故を
17 // 防ぐためリセットする。performer.state を WaitStart に戻すことで、新マスタ
18 // の
19 // 最初の BEAT を受信してから演奏を再開する (リセット直後の不安定な期間を飛ば
20 // す)。
21 void resetAfterMasterReset(SystemData& data) {
22     data.receiver.pending.valid = false;
23     data.receiver.hasFirstBeat  = false;
24     data.receiver.lastBeatNo    = 0;
25     data.performer.state        = PerformerState::WaitStart;
26 }
27
28 // namespace
29
30 // 時計同期: min フィルタ (窓内最大 offset サンプル追従、NTP 系の定石)。
31 // 戻り値: スナップ (大ジャンプの即時追従) が起きたか。
32 //
33 // 設計意図 (tools/verification/results/
34 //   MOP5_systematic_shift_analysis_20260710.md Sec.4/Sec.8 案4):
35 //   SoftAP のマルチキャストは DTIM バッファリングで 204.8ms 周期のバースト配
36 //   送になり、
37 //   サンプル (= timestampMs - millis() = 真のオフセット  $\theta$  - 配送遅延 D)
38 //   は
39 //   「 $\theta$  から 0~205ms 下に散らばる」。旧 EMA ( $\alpha=0.20$ ) は新鮮な CTRL と
40 //   ~102ms 古い
41 //   BEAT の平均的な位置に落ち着き、推定マスタ時計が真値より 40~55ms 遅れて
42 //   いた。
43 //   min フィルタは配送遅延が最小のサンプル (= 最大 sample) だけを信じるた
44 //   め、
45 //   推定時計が最小遅延線に張り付き、この系統遅れがほぼ消える。

```

```

36 //
37 // 実装:
38 //   - sample > 現 offset なら即採用 (より遅延の小さいサンプル。UNO R4 の
      millis() が
39 //   指揮者比で遅い個体 = sample が上昇トレンドの場合の追従もこの経路で即時
      )。
40 //   - 同時に窓 (clockSyncWindowMs) 内の最大 sample を記録し、窓満了で offset
      を
41 //   窓内最大値へ引き直す。offset が上振れノイズや下降トレンド (楽器側時計
      が
42 //   速い個体) で古くなっても、窓長ぶんの遅れで必ず追従し直せる。
43 //   - スナップ: 指揮者リセットでマスタ時計 millis() が 0 付近へ巻き戻ると
      sample が
44 //   数十秒~数分ぶん飛ぶ。|sample - 現 offset| が snapThresholdMs を超え
      たら
45 //   フィルタをやめて即時採用し、収束カウントと窓をやり直す (旧 EMA 実装と
      同じ
46 //   1 パケット追従を維持)。SoftAP 直結 LAN の正常遅延 (バースト待ち含め
      ~205ms)
47 //   では閾値 1000ms に届かない。
48 bool OrcReceiverModule::updateClockOffset(SystemData& data, uint32_t
      timestampMs) {
49     const int32_t sample = (int32_t)(timestampMs - millis());
50     bool snapped = false;
51     if (data.sync.sampleCount == 0) {
52         data.sync.offsetMs = sample;
53         winValid_ = false;
54     } else {
55         const int32_t jump = sample - data.sync.offsetMs;
56         if (jump > (int32_t)cfg_.clockSyncSnapThresholdMs ||
57             jump < -(int32_t)cfg_.clockSyncSnapThresholdMs) {
58             // スナップ: 巻き戻った (または大きく飛んだ) マスタ時計に即追従
              し、
59             // 収束カウントもやり直す。
60             data.sync.offsetMs = sample;
61             data.sync.sampleCount = 0;
62             data.sync.converged = false;
63             winValid_ = false;
64             snapped = true;
65         } else {
66             if (sample > data.sync.offsetMs) {
67                 data.sync.offsetMs = sample;
68             }
69             const uint32_t nowMs = millis();
70             if (!winValid_) {
71                 winValid_ = true;
72                 winStartMs_ = nowMs;
73                 winMaxSample_ = sample;

```

```

74         } else {
75             if (sample > winMaxSample_) winMaxSample_ = sample;
76             if (nowMs - winStartMs_ >= cfg_.clockSyncWindowMs) {
77                 // 窓満了：窓内最大サンプルへ引き直し（下方方向の再追従）、
78                 //  次の窓を開く
79                 data.sync.offsetMs = winMaxSample_;
80                 winValid_ = false;
81             }
82         }
83     }
84     if (data.sync.sampleCount < 0xFFFF) data.sync.sampleCount++;
85     if (data.sync.sampleCount >= cfg_.clockSyncMinSamples) data.sync.
86         converged = true;
87     return snapped;
88 }
89 void OrcReceiverModule::updateInput(SystemData& data) {
90     #if MOP_TEST == 4 || MOP_TEST == 5
91         // ループストール検出（ハートビート）：本モジュールの updateInput は 3
92         // フェーズループの
93         // 毎周期呼ばれる（名目 loopIntervalMs=2ms）ので、前回呼び出しからの間隔
94         // が閾値を超えたら
95         // 「ループが回っていなかった区間」として 1 行記録する。7/10-11 の再計測
96         // で、バースト到着から
97         // 位相 ~120~165ms の区間で発火が消える周期ストール（発火 lateMs p95 47
98         // ms・MOP4 尾悪化の
99         // 共通原因）が観測されており、その粒度（単一の長ブロックか短いブロックの
100         // 連なりか）と正確な
101         // 窓位置を確定するための計測（MOP5_countermeasure_eval_20260710.md Sec
102         // .3.3/Sec.5(b)-2）。
103         // 形式：M45S,<partId>,<deviceMs>,<gapMs>（deviceMs = ストール明け = 今
104         // 回呼び出しの millis()）
105         // 閾値 10ms：名目 2ms + 実測の健常ループ粒度（発火 late p95 9ms）を上回
106         // る異常だけを拾い、
107         // シリアル帯域を守る（ストールは ~5 回/秒 なので 1 行 ~25B x 5 = 帯域の
108         // 1% 未満）。
109     {
110         static uint32_t sLastLoopMs = 0;
111         const uint32_t nowMs = millis();
112         if (sLastLoopMs != 0) {
113             const uint32_t gapMs = nowMs - sLastLoopMs;
114             if (gapMs >= 10) {
115                 mop_test::mprintf("M45S,%u,%lu,%lu\n",
116                                     (unsigned)cfg_.partId,
117                                     (unsigned long)nowMs,
118                                     (unsigned long)gapMs);
119             }
120         }
121     }
122     #endif
123 }

```

```

111     }
112     sLastLoopMs = nowMs;
113 }
114 #endif
115 // 1. CTRL を受信していれば時計同期 + 状態取り込み
116 if (data.orcNet.hasNewCtrl) {
117     if (updateClockOffset(data, data.orcNet.lastCtrl.header.timestampMs))
118     {
119         resetAfterMasterReset(data);
120     }
121
122     data.ctrl.bpm = data.orcNet.lastCtrl.payload.bpmQ8 / 8.0f;
123     data.ctrl.velocity = data.orcNet.lastCtrl.payload.velocity;
124     data.ctrl.state = data.orcNet.lastCtrl.payload.state;
125     // production ゲームモード: 予約バイトから UI 状態を展開 (
126     //   UiRelayModule が PC へ中継)
127     data.ctrl.bpmQ8 = data.orcNet.lastCtrl.payload.bpmQ8;
128     data.ctrl.mode = data.orcNet.lastCtrl.payload.mode;
129     data.ctrl.navCursor = data.orcNet.lastCtrl.payload.navCursor;
130     data.ctrl.targetBpm = data.orcNet.lastCtrl.payload.targetBpm;
131     data.ctrl.score = data.orcNet.lastCtrl.payload.score;
132     data.ctrl.lastReceivedMs = millis();
133 }
134
135 // 2. BEAT 受信:
136 //   - 連送 (beatRedundancy 回) で同一 beatNo が複数届くが、payload は完
137 //     全同一。
138 //   - 違うのは header.timestampMs (各送信時刻) と header.seq。
139 //   - pending (発音予約) は初到着の 1 個だけ採用。同 beatNo の発火後に
140 //     後着が再キューされて「同じ拍を二度発音」する事故を避ける。
141 //   - clock sync は連送 4 個ぶん全部サンプルとして使う。min フィルタでは
142 //     重複は
143 //   - 自然に無害: 同一 timestampMs で millis() が経過ぶん進むため sample
144 //     は
145 //   - 初回以下になり、窓内最大値を押し上げない (旧 EMA の重複用  $\alpha$  は不
146 //     要になった)。
147 if (data.orcNet.hasNewBeat) {
148     const uint16_t bn = data.orcNet.lastBeat.payload.beatNo;
149     bool isDuplicate = data.receiver.hasFirstBeat &&
150         (bn == data.receiver.lastBeatNo);
151
152     if (updateClockOffset(data, data.orcNet.lastBeat.header.timestampMs))
153     {
154         // この BEAT 自体は新マスタ時計の正しい playAtMasterMs を持つの
155         //   で、
156         // 重複扱いを解いて「新時計の最初の BEAT」として受理し直す。
157         resetAfterMasterReset(data);
158         isDuplicate = false;
159     }
160 }

```

```

151     }
152
153     if (!isDuplicate) {
154         data.receiver.pending.valid = true;
155         data.receiver.pending.beatNo = bn;
156         data.receiver.pending.playAtMasterMs =
157             data.orcNet.lastBeat.payload.playAtMasterMs;
158         data.receiver.pending.enqueueAtMs = millis();
159         data.receiver.lastBeatNo = bn;
160         data.receiver.hasFirstBeat = true;
161
162         #if MOP_TEST == 4 || MOP_TEST == 5
163             // MOP4/MOP5 共通の BEAT 受信記録（受理した初回のみ = 1 beat 1
164             // record）。
165             // 旧実装は main.cpp 側にも M5I 出力があり同一拍が二重記録されて
166             // いたため、
167             // 計測出力はこの 1 箇所に統一した（発火時の M45F は applyPattern
168             // .cpp）。
169             // localMasterMs = 受信処理時点の推定マスタ時刻。集計側が
170             // lateMs = max(0, localMasterMs - playAtMasterMs) を計算し、
171             // beatLookahead の発音予約が受信時点で間に合っているかを判定す
172             // る。
173             {
174                 const uint32_t tLocal = millis();
175                 const uint32_t localMasterMs = tLocal + (uint32_t)data.sync.
176                     offsetMs;
177                 mop_test::mprintf("M45R,%u,%u,%lu,%lu,%ld,%lu\n",
178                     (unsigned)cfg_.partId, (unsigned)bn,
179                     (unsigned long)data.orcNet.lastBeat.payload
180                     .playAtMasterMs,
181                     (unsigned long)tLocal,
182                     (long)data.sync.offsetMs,
183                     (unsigned long)localMasterMs);
184             }
185         #endif
186
187         #if MOP_TEST == 9
188             mop_test::mprintf("M9,%u,%u,%lu,%lu\n",
189                 (unsigned)cfg_.partId, (unsigned)bn,
190                 (unsigned long)data.orcNet.lastBeat.header.seq,
191                 (unsigned long)millis());
192         #endif
193     }
194     // 重複でも lastBeatMs は更新する（受信タイムアウト監視・診断ログ用）
195     data.receiver.lastBeatMs = millis();
196 }
197 }

```

リスト 5 楽器ノードの発音予約発火と輪唱進行 (node_02/src/applyPattern.cpp)

```

1 // Build / Upload / Monitor (run from project root):
2 //   pio run -d firmware/production/node_02
3 //   pio run -d firmware/production/node_02 -t upload
4 //   pio device monitor -d firmware/production/node_02
5 //
6 // 楽器ノード（輪唱の 1 声部）の判断ロジック
7 // 設計方針: BPM はあくまで補助（durationMs 計算で使用）。score の進行は
8 // 「指揮者の拍番号 firedBeatNo」で決める。輪唱は headRestBeats だけ頭の拍を
9 // 読み飛ばしてから kScore[0] を鳴らし始める（声部ごとにずれて入る）。
10 // 周回は CANON_CYCLE_BEATS（曲長 + 最終声部の遅延）のサイクル窓で管理し、
11 // 最終声部（node_05）が 1 周を終えるまで先頭声部は次の周回を始めない。
12 // 拍番号駆動なので、Processing をいつ起動しても「曲の現在位置」から自然に鳴
    る。
13 //
14 // 処理フロー:
15 //   1. 時計オフセット更新（受信側で済 - Receiver が data.sync を更新）
16 //   2. 演奏状態遷移（Idle -> WaitStart -> Playing）
17 //   3. 保留 BEAT のキューイング（受信側で済 - data.receiver.pending）
18 //   4. マスタ時刻判定（発音粒度）
19 //   5. 楽譜進行: firedBeatNo と headRestBeats から kScore のインデックスを算
    出して発火
20 //   6. velocity 合成（fireScoreEvent 内）
21 #include <Arduino.h>
22
23 #include "SystemData.h"
24 #include "ProjectConfig.h"
25 #include "SerialDebug.h"
26 #include "MopTest.h"
27 #include "score_data.h"
28
29 namespace {
30
31 // durationQ8 を ms に変換（Q8: 256 = 1 拍）
32 // BPM は CTRL から受け取った参考値。NoteOff の予定時刻計算に使う補助。
33 uint16_t durationQ8ToMs(uint16_t durationQ8, float bpm) {
34     const float beats = (float)durationQ8 / 256.0f;
35     const float ms = beats * 60000.0f / (bpm < 10.0f ? logic_params::
        DEFAULT_BPM : bpm);
36     return (uint16_t)(ms > 60000.0f ? 60000.0f : ms);
37 }
38
39 // 楽譜の 1 イベントを発火（NoteOn 予約のみ）。
40 // 消音は Processing 側が NotePacket.durationMs から自動で行うため、ここでは
41 // noteIsSounding 等の追跡は不要。
42 // 細分音符（subNote != 0）があれば、現在 BPM から ms を計算して予約スロット
    に積む。
43 void fireScoreEvent(SystemData& data, const ScoreEvent& ev, uint32_t now) {
44     const bool isRest = (ev.flags & 0x04) != 0 || ev.noteNumber == 0;

```



```

45     if (!isRest) {
46         // velocity 合成: score x ctrl / 127
47         uint16_t v = ((uint16_t)ev.velocity * (uint16_t)data.ctrl.velocity) /
48             127;
49         if (v > 127) v = 127;
50
51         const uint16_t durMs = durationQ8ToMs(ev.durationQ8, data.ctrl.bpm);
52         data.noteOut.noteNumber = ev.noteNumber;
53         data.noteOut.velocity = (uint8_t)v;
54         data.noteOut.durationMs = durMs;
55         data.noteOut.pendingOn = true;
56     }
57
58     // 細分音符の予約 (拍頭から subOffsetQ8/256 拍ぶん遅らせて発火)
59     if (ev.subNote != 0) {
60         const float bpm = (data.ctrl.bpm >= 1.0f) ? data.ctrl.bpm
61             : logic_params::DEFAULT_BPM
62             ;
63         const float beats = (float)ev.subOffsetQ8 / 256.0f;
64         const uint32_t subDelayMs = (uint32_t)(beats * 60000.0f / bpm);
65         data.score.pendingSub = true;
66         data.score.pendingSubAtMs = now + subDelayMs;
67         data.score.pendingSubNote = ev.subNote;
68         data.score.pendingSubVelocity = ev.subVelocity;
69         data.score.pendingSubDurationMs = durationQ8ToMs(ev.subDurationQ8,
70             bpm);
71     }
72 }
73
74 // 予約された細分音符の時刻が来ていれば NoteOn を出す。
75 // applyPattern の先頭で呼び、楽譜進行 (4 分 BEAT 受信) と同一ループに重なら
76 // ないようにする。
77 // noteOutSub (専用スロット) に書く。noteOut (メイン) とは独立なので、
78 // 同一ループ反復で fireScoreEvent が noteOut を書いても衝突しない。
79 void firePendingSub(SystemData& data, uint32_t now) {
80     if (!data.score.pendingSub) return;
81     if ((int32_t)(now - data.score.pendingSubAtMs) < 0) return;
82
83     uint16_t v = ((uint16_t)data.score.pendingSubVelocity *
84         (uint16_t)data.ctrl.velocity) / 127;
85     if (v > 127) v = 127;
86     data.noteOutSub.noteNumber = data.score.pendingSubNote;
87     data.noteOutSub.velocity = (uint8_t)v;
88     data.noteOutSub.durationMs = data.score.pendingSubDurationMs;
89     data.noteOutSub.pendingOn = true;
90     DBG_PRINTF("[SUB]_note=%u_vel=%u_dur=%u_delay=%lu_now=%lu\n",
91         (unsigned)data.score.pendingSubNote,
92         (unsigned)v,

```

```

89         (unsigned)data.score.pendingSubDurationMs,
90         (unsigned long)(now - data.score.pendingSubAtMs),
91         (unsigned long)now);
92     data.score.pendingSub = false;
93 }
94
95 void updatePerformerLed(SystemData& data) {
96     using namespace logic_params;
97     switch (data.performer.state) {
98         case PerformerState::Idle:
99             data.led.solidOn = false;
100             data.led.blinkIntervalMs = LED_IDLE_MS;
101             break;
102         case PerformerState::WaitStart:
103             data.led.solidOn = false;
104             data.led.blinkIntervalMs = LED_WAIT_START_MS;
105             break;
106         case PerformerState::Playing:
107             data.led.solidOn = true;
108             break;
109     }
110 }
111
112 uint32_t sLastBeatRecvMs = 0;
113
114 } // namespace
115
116 void applyPattern(SystemData& data) {
117     using namespace logic_params;
118     const uint32_t now = millis();
119
120     // 0. 予約された細分音符（8 分音符等）の発火判定。
121     // 前ループまでに fireScoreEvent から積まれた予約が、現時刻に達してい
122     // れば発音する。
123     firePendingSub(data, now);
124
125     // 2. 演奏状態遷移（Idle -> WaitStart -> Playing）
126     // Playing への遷移条件は「最初の BEAT を受信した」だけ（sync.converged
127     // 待ちで
128     // 鳴らない症状を避ける）。輪唱の「頭の休符」は Playing には入ってから
129     // headRestBeats
130     // ぶん拍を消費する形で表すので、ここでは入り拍を待たない。
131     switch (data.performer.state) {
132         case PerformerState::Idle:
133             if (data.orcNet.wifiConnected) {
134                 data.performer.state = PerformerState::WaitStart;
135             }
136             break;

```

```

134     case PerformerState::WaitStart:
135         if (data.receiver.hasFirstBeat) {
136             data.performer.state = PerformerState::Playing;
137         }
138         // リセット復帰: BEAT 未受信でも CTRL が Conducting (state==2) なら
139         // Playing に合流する。次の BEAT が来るまで鳴らないが、到着時に即
140         // 発音できる。
141         else if (data.ctrl.state == 2 && data.ctrl.lastReceivedMs > 0) {
142             data.performer.state = PerformerState::Playing;
143             sLastBeatRecvMs = now;
144         }
145         break;
146     case PerformerState::Playing:
147         // 指揮者からの BEAT が 10 秒以上途絶えたら待機に戻して LED を点
148         // 滅に切り替える
149         if (sLastBeatRecvMs > 0 && (now - sLastBeatRecvMs) > 10000) {
150             data.performer.state = PerformerState::WaitStart;
151             data.receiver.hasFirstBeat = false;
152             sLastBeatRecvMs = 0;
153         }
154         break;
155     }
156
157     // 4. 発音判定: マスタ時刻 playAtMasterMs に揃えて発火する (本来の同期設
158     // 計)。
159     //   targetLocalMs = playAtMasterMs - sync.offsetMs (マスタ時刻 → 自分の
160     //   ローカル ms)
161     //   - waitMs <= 0 (既に到来 or 受信遅延): 即発火 (期限切れでも捨てない。
162     //   捨てると鳴らなくなる)
163     //   - waitMs > 0: この周期は何もしない。次ループで再評価し時刻到来した
164     //   ら発火する
165     // 各スレーブが同じ playAtMasterMs に対してそれぞれの sync.offsetMs を引
166     // いて
167     // 待つため、複数スレーブの発音はマスタ時刻基準で自然に揃う。
168     // 旧即発火版は受信遅延ぶんスレーブ間でズレていたが、本実装ではそのズレが
169     // 消える。
170
171     bool        fired        = false;
172     uint16_t    firedBeatNo = 0; // 発火した拍の指揮者拍番号 (1 始まり)
173     if (data.performer.state == PerformerState::Playing &&
174         data.receiver.pending.valid) {
175         const int32_t targetLocalMs =
176             (int32_t)data.receiver.pending.playAtMasterMs - data.sync.
177                 offsetMs;
178         const int32_t waitMs = targetLocalMs - (int32_t)now;
179         if (waitMs <= 0) {
180             fired        = true;
181             firedBeatNo = data.receiver.pending.beatNo;
182         }
183     }

```

```

172 #if MOP_TEST == 4 || MOP_TEST == 5
173     // MOP4/MOP5: 発火時記録（発音予約が実際に発火した瞬間のデバイス
174     //     側計測）。
175     // localMasterMs = 発火時点の推定マスタ時刻。同一 beatNo の
176     //     localMasterMs の
177     // ノード間レンジが MOP4（楽器間同期誤差）、playAtMasterMs に対す
178     //     る遅刻
179     // lateMs = max(0, localMasterMs - playAtMasterMs) が MOP5（予約
180     //     遅刻）。
181     // USB 受信時刻に依存しない。集計は tools/verification/scripts/
182     //     の
183     // mop4_sync_error.py / mop5_comm_delay.py。
184     {
185         const uint32_t localMasterMs = now + (uint32_t)data.sync.
186             offsetMs;
187         mop_test::mprintf("M45F,%u,%u,%lu,%lu,%ld,%lu\n",
188             (unsigned)ORC_RECEIVER_CONFIG.partId,
189             (unsigned)firedBeatNo,
190             (unsigned long)data.receiver.pending.
191                 playAtMasterMs,
192             (unsigned long)now,
193             (long)data.sync.offsetMs,
194             (unsigned long)localMasterMs);
195     }
196 #endif
197     data.receiver.pending.valid = false;
198     sLastBeatRecvMs = now;
199 }
200 // waitMs > 0: 次ループで再評価（pending は valid のまま残す）
201 }
202
203 // 5. 楽譜進行: 指揮者拍番号 firedBeatNo から自分の楽譜インデックスを算出
204 //     する。
205 // 輪唱サイクル方式（4 声対応）: cyclePos = (firedBeatNo - 1) %
206 //     CANON_CYCLE_BEATS
207 //     cyclePos < headRestBeats : 自分の入り前（頭の休符）→
208 //     鳴らさない
209 //     headRest <= cyclePos < headRest+曲長 : local = cyclePos -
210 //     headRestBeats で発火
211 //     cyclePos >= headRest+曲長 : 自分の 1 周は終了 → サイ
212 //     クル末尾まで休む
213 // サイクル長 56 拍 = 曲長 32 + 最終声部（node_05, headRest=24）の遅
214 //     延。全声部が
215 // 同じ周期を共有するため、最終声部が 1 周を終えるまで先頭声部は次の周
216 //     回を
217 // 始めない（旧 % kScoreLength 方式は各声部が勝手に周回し輪唱の終端が崩
218 //     れた）。
219 // 拍番号で引くので、Processing をいつ起動しても曲の現在位置から鳴り始め

```

```

205     る。
        // 拍が一つ飛んでも firedBeatNo に追従するだけでズレが残らない（自己補正）
        。
206     if (fired && kScoreLength > 0) {
207         const uint32_t cyclePos =
208             (uint32_t)(firedBeatNo - 1) % (uint32_t)CANON_CYCLE_BEATS;
209         const int32_t local =
210             (int32_t)cyclePos - (int32_t)ORC_RECEIVER_CONFIG.headRestBeats;
211         if (local >= 0 && (uint32_t)local < (uint32_t)kScoreLength) {
212             fireScoreEvent(data, kScore[local], now);
213             data.score.currentEventIndex = (uint16_t)local;    // 診断ログ用
214         }
215     }
216
217     // (NoteOff 判定はない: 消音は Processing 側が NotePacket.durationMs から
        自動で行う。)
218
219     // LED
220     updatePerformerLed(data);
221 }

```

リスト 6 指揮者ノードの拍検出・テンポ推定・状態遷移 (node_01/src/applyPattern.cpp)

```

1 // Build / Upload / Monitor (run from project root):
2 //   pio run -d firmware/production/node_01
3 //   pio run -d firmware/production/node_01 -t upload
4 //   pio device monitor -d firmware/production/node_01
5 //
6 // 指揮者ノードの判断ロジック
7 // 仕様 Sec.2.4.2.4 の処理フロー:
8 //   1. IIR LPF + ノルム計算
9 //   2. 拍検出 (Conducting 時のみ)
10 //   3. テンポ推定 (EMA)
11 //   4. 状態遷移 (Idle -> Calibrating -> Conducting <-> Fallback)
12 #include <math.h>
13 #include <Arduino.h>
14
15 #include "SystemData.h"
16 #include "ProjectConfig.h"
17 #include "SerialDebug.h"
18
19 namespace {
20
21 float    sLpfAcc[3] = {0, 0, 0};
22 bool     sLpfInit = false;
23 uint32_t sLastBeatMs = 0;
24 // 初期テンポ 100 BPM。1 音目 (= 最初の拍) はこの値で CTRL を流す。
25 // 2 拍目で sBpmInit が false なので「1→2 拍目の間隔」をそのまま簡易テンポと

```

```

    して採用し、
26 // 3 拍目以降は BPM_EMA_ALPHA で随時補正する。
27 float    sBpmEma = 100.0f;
28 bool     sBpmInit = false;
29
30 // 拍検出のゲートステート。
31 // Idle : 動加速度が小さい状態。積分しない（ノイズ蓄積なし）。
32 // Armed : dynNorm > BEAT_DYN_THRESHOLD_G で突入。Armed 中だけ
33 //         dynAcc を積分して経路長 sPathLen を計算する。
34 // 拍は Armed -> Idle のリリースエッジで判定する (= 振り終わりタイミング)。
35 enum class BeatGate : uint8_t { Idle, Armed };
36
37 BeatGate sGate          = BeatGate::Idle;
38 float    sVel[3]        = {0, 0, 0};    // 動加速度の時間積分 (m/s) —
    Armed 中のみ更新
39 float    sPathLen       = 0.0f;          // |sVel| の時間積分 (m) —
    Armed 中のみ更新
40 float    sArmedPeakDyn   = 0.0f;          // Armed 中の dynNorm 最大値 (デ
    バッグ用)
41 uint32_t sArmedAtMs      = 0;             // Armed に入った時刻
42 uint32_t sLastImuMs      = 0;             // 直前に積分した IMU サンプル時刻
43 bool     sBeatFiredInArmed = false;       // 現 Armed セッション中に既に発火
    したか
44 uint32_t sReleaseStartMs = 0;             // リリース条件が連続成立し始めた
    時刻 (0=未成立)

45
46 void gateToIdle() {
47     sGate          = BeatGate::Idle;
48     sVel[0] = sVel[1] = sVel[2] = 0.0f;
49     sPathLen       = 0.0f;
50     sArmedPeakDyn   = 0.0f;
51     sArmedAtMs      = 0;
52     sLastImuMs      = 0;
53     sBeatFiredInArmed = false;
54     sReleaseStartMs = 0;
55 }
56
57 // テンポ推定のリセット (Menu からの演奏セッション開始時に呼ぶ)。
58 // sLastBeatMs=0 で次の拍を「1 拍目」(間隔計算なし)に戻す。これをしないと
59 // Menu 滞在時間が拍間隔として EMA に混入し、BPM_MIN クランプ値 (40) に
60 // 引っ張られた BPM を新セッションの頭で数拍引きずる。
61 void resetTempoTracking(SystemData& data) {
62     sLastBeatMs = 0;
63     sBpmEma     = 100.0f;
64     sBpmInit    = false;
65     data.tempo.bpm = 100.0f;
66     data.tempo.nextBeatPredictedMs = 0;

```

```

67 }
68
69 // Fallback に落ちる直前の状態 (= 復帰先)。Menu/Result で IMU が止まっても
70 // 復帰後に元の画面へ戻る。既定 Menu は「Calibrating 完了前に Fallback は
71 // 起きない」ため実際には使われない保険値。
72 ConductorState sStateBeforeFallback = ConductorState::Menu;
73
74 // —— production: メニューナビのゲート (拍検出ゲートとは独立) ——
75 // 重力基準の縦/横判定。加速度の遅い LPF で重力ベクトルを推定し、振り加速度
76 // (accLpf - 推定重力) を「重力軸成分」と「水平面成分」に分解する。センサに
77 // 取り付け角度がついていても「地面に対して垂直=縦 (決定) / 水平=横 (カーソル
78 // になる (旧方式のセンサ軸直読みは取り付け角度で破綻した)。
79 // 瞬時値は振り始めの向きが暴れるので、Armed 中の判定窓で両成分を積算し、
80 // 窓終了かリリースの早い方で 1 回だけ判定・発火する。Menu / Result 状態でのみ動かす。
81 enum class NavGate : uint8_t { Idle, Armed };
82 NavGate sNavGate = NavGate::Idle;
83 uint32_t sLastNavMs = 0; // 最後に発火した時刻 (不応期の基準)
84 uint32_t sNavArmedAtMs = 0; // Armed 突入時刻 (判定窓の基準)
85 bool sNavFired = false; // 現 Armed セッションで発火済みか
86 float sNavVertAccum = 0.0f; // |重力軸成分| の積算
87 float sNavHorizAccum = 0.0f; // |水平面成分| の積算
88 float sNavHorizVec[3] = {0, 0, 0}; // 水平面成分ベクトルの積算 (カーソル移動方向用)
89 bool sNavNeedsRelease = false; // 横振り発火後、dynNorm < NAV_RELEASE_G で解除されるまで再発火を禁止
90
91 // ナビ用の重力ベクトル推定 (accLpf をさらに遅い LPF に通したものの)。
92 // 拍検出のキャリブ値 gravityMag はスカラーなので方向の分解には使えない。
93 float sNavGrav[3] = {0, 0, 0};
94 bool sNavGravInit = false;
95
96 // 1 振りを 1 操作として処理する。横振り→カーソル移動 (data.game.navCursor を更新・false)、
97 // 縦振り→決定 (true を返す)。多重発火は Armed ゲート + 発火済みフラグ + 不応期で防ぐ。
98 bool updateNav(SystemData& data, uint32_t now, uint8_t itemCount) {
99     using namespace logic_params;
100     if (!data.imu.ready) return false;
101
102     if (sNavGate == NavGate::Idle) {
103         // 横振り発火後、ユーザーが振りを止める (dynNorm < release) まで再突入を禁止。
104         // これにより 1 回の物理的な振りから 1 回だけ発火し、LPF 尾引きや
105         // 強制 Idle→即再 Armed で同じ振り中に 2 回トグルする問題を防ぐ。
106         if (sNavNeedsRelease) {

```

```

107         if (data.imu.dynNorm < NAV_RELEASE_G) sNavNeedsRelease = false;
108         return false;
109     }
110     // しきい値超え + 不応期経過で Armed へ (積算を開始)
111     if (data.imu.dynNorm <= NAV_SWING_THRESHOLD_G) return false;
112     if ((now - sLastNavMs) < NAV_REFRACTORY_MS) return false;
113     sNavGate = NavGate::Armed;
114     sNavArmedAtMs = now;
115     sNavFired = false;
116     sNavVertAccum = 0.0f;
117     sNavHorizAccum = 0.0f;
118     sNavHorizVec[0] = sNavHorizVec[1] = sNavHorizVec[2] = 0.0f;
119 }
120
121 // Armed 中: 振り加速度を重力軸成分と水平面成分に分解して積算する
122 const float gravNorm = sqrtf(sNavGrav[0] * sNavGrav[0] +
123                               sNavGrav[1] * sNavGrav[1] +
124                               sNavGrav[2] * sNavGrav[2]);
125 if (gravNorm > 1e-3f) {
126     const float invG = 1.0f / gravNorm;
127     float swing[3]; // 振り加速度ベクトル (g 単位、重力差し引き済み)
128     for (int i = 0; i < 3; ++i) swing[i] = data.imu.accLpf[i] - sNavGrav[i];
129     const float vert = (swing[0] * sNavGrav[0] +
130                        swing[1] * sNavGrav[1] +
131                        swing[2] * sNavGrav[2]) * invG; // 符号付き重力
132                                     軸成分
133     sNavVertAccum += fabsf(vert);
134     float horizSq = 0.0f;
135     for (int i = 0; i < 3; ++i) {
136         const float h = swing[i] - vert * sNavGrav[i] * invG; // 水平面
137                                     成分
138         sNavHorizVec[i] += h;
139         horizSq += h * h;
140     }
141     sNavHorizAccum += sqrtf(horizSq);
142 }
143
144 // 判定: 窓終了かリリースの早い方で 1 回だけ発火
145 const bool windowDone = (now - sNavArmedAtMs) >= NAV_DECISION_WINDOW_MS;
146 const bool release = data.imu.dynNorm < NAV_RELEASE_G;
147 bool decide = false;
148 if (!sNavFired && (windowDone || release)) {
149     sNavFired = true;
150     sLastNavMs = now;
151     if (sNavVertAccum >= sNavHorizAccum * NAV_VERT_DOMINANCE) {
152         decide = true; // 縦振りが支配的 → 決定
153     } else {

```



```

152 // 横振りが支配的 → カーソル移動
153 if (itemCount == 2) {
154     // 2 項目の場合は方向に依存せずトグル。左右どちらに振っても
155     // 毎回モードが切り替わる。方向判定は重力分解の精度や取り付け
156     // 角度の影響で不安定になりやすく、端に張り付いて動かなくなる
157     // 問題があったため廃止。
158     data.game.navCursor = 1 - data.game.navCursor;
159     sNavNeedsRelease = true;
160 } else if (itemCount > 2) {
161     int dom = 0;
162     for (int i = 1; i < 3; ++i) {
163         if (fabsf(sNavHorizVec[i]) > fabsf(sNavHorizVec[dom]))
164             dom = i;
165     }
166     const float dir = sNavHorizVec[dom] * NAV_LR_SIGN;
167     if (dir < 0.0f) { if (data.game.navCursor > 0)
168         data.game.navCursor--; }
169     else { if (data.game.navCursor + 1 < itemCount)
170         data.game.navCursor++; }
171     sNavNeedsRelease = true;
172 }
173 data.sender.forceCtrlSend = true;
174 }
175 DBG_PRINTF("[N1 NAV vert=%5.2f horiz=%5.2f->%s]\n",
176     sNavVertAccum, sNavHorizAccum,
177     decide ? "DECIDE" : "CURSOR");
178 }
179 // 振りが収まったら (release) 次操作に備える。
180 // 発火済みで不応期を越えた場合は release を待たず強制的に Idle へ戻す。
181 // dynNorm が NAV_RELEASE_G と NAV_SWING_THRESHOLD_G の間に留まって Armed
182 // から
183 // 抜けられなくなり、2 回目以降の振りを検知できなくなる問題を修正。
184 if (release) {
185     sNavGate = NavGate::Idle;
186 } else if (sNavFired && (now - sLastNavMs) >= NAV_REFRACTORY_MS) {
187     sNavGate = NavGate::Idle;
188 }
189 return decide;
190 }
191 // —— production: 状態遷移直後のデッドタイム (ジェスチャ/拍検出の不感時間)
192 ——
193 // 遷移を起こした振りの残り (惰性・戻し) を次状態の入力として拾わないため、
194 // 遷移から STATE_TRANSITION_DEADTIME_MS の間は拍検出とナビを無視する。
195 ConductorState sPrevState = ConductorState::Idle;
196 uint32_t sStateEnteredMs = 0;
197 // 状態遷移を検知したら不感時間を開始し、拍/ナビ両ゲートの撃ちかけを破棄す

```

```

    る。
195 // 遷移は状態遷移 switch 内と拍検出ブロック内 (Conducting→Result) の複数箇所
    で
196 // 起きるため、switch の前後 2 箇所から呼ぶ (state 不変なら何もしない幕等処理
    )。
197 void noteStateTransition(SystemData& data, uint32_t now) {
198     if (data.conductor.state == sPrevState) return;
199     sPrevState      = data.conductor.state;
200     sStateEnteredMs = now;
201     gateToIdle();
202     sNavGate = NavGate::Idle;
203     sNavNeedsRelease = false;
204     data.beat.pathLenM = 0.0f;
205     // 状態遷移直後に CTRL を即送信し、PC 側の画面切替を高速化する
206     data.sender.forceCtrlSend = true;
207 }
208
209 bool inTransitionDeadttime(uint32_t now) {
210     return (now - sStateEnteredMs) < logic_params::
        STATE_TRANSITION_DEADTIME_MS;
211 }
212
213 // メトロノームガイド強度 (経過拍ベースの固定スケジュール・通信不要)。1.0=
    はっきり / 0.0=なし。
214 // 採点重みは 1 - この値 (ガイドが薄い/無い区間ほど重く配点する)。
215 float gameGuideIntensity(uint16_t beatCount) {
216     using namespace logic_params;
217     if (beatCount < GAME_GUIDE_FULL_BEATS) return 1.0f;
218     if (beatCount >= GAME_GUIDE_ZERO_BEATS) return 0.0f;
219     const float span = (float)(GAME_GUIDE_ZERO_BEATS - GAME_GUIDE_FULL_BEATS)
        ;
220     return 1.0f - (float)(beatCount - GAME_GUIDE_FULL_BEATS) / span;
221 }
222
223 void updateLed(SystemData& data) {
224     using namespace logic_params;
225     switch (data.conductor.state) {
226     case ConductorState::Idle:
227         data.led.solidOn = false;
228         data.led.blinkIntervalMs = LED_IDLE_MS;
229         break;
230     case ConductorState::Calibrating:
231         data.led.solidOn = false;
232         data.led.blinkIntervalMs = LED_CALIBRATING_MS;
233         break;
234     case ConductorState::Conducting:
235         // ゲーム中でガイドが残っている区間は目標テンポで点滅 (LED メトロ
            ノームガイド)。

```

```

236 // ガイドが切れたら点灯のまま（自由演奏も常時点灯）。
237 if (data.game.mode == 1 && data.game.targetBpm > 0 &&
238     gameGuideIntensity(data.game.gameBeatCount) > 0.0f) {
239     data.led.solidOn = false;
240     data.led.blinkIntervalMs = (uint16_t)(30000u / data.game.
        targetBpm);
241 } else {
242     data.led.solidOn = true;
243 }
244 break;
245 case ConductorState::Fallback:
246     data.led.solidOn = false;
247     data.led.blinkIntervalMs = LED_FALLBACK_MS;
248     break;
249 case ConductorState::Menu:
250     data.led.solidOn = false;
251     data.led.blinkIntervalMs = LED_MENU_MS;
252     break;
253 case ConductorState::Result:
254     data.led.solidOn = false;
255     data.led.blinkIntervalMs = LED_RESULT_MS;
256     break;
257 }
258 }
259
260 } // namespace
261
262 void applyPattern(SystemData& data) {
263     using namespace logic_params;
264     const uint32_t now = millis();
265
266     // 1. IIR LPF + ノルム計算 + 動加速度ノルム（静止ノルム補正後）の計算
267     if (data.imu.ready) {
268         if (!sLpfInit) {
269             for (int i = 0; i < 3; ++i) sLpfAcc[i] = data.imu.acc[i];
270             sLpfInit = true;
271         } else {
272             for (int i = 0; i < 3; ++i) {
273                 sLpfAcc[i] = (1.0f - LPF_ALPHA) * sLpfAcc[i] +
274                     LPF_ALPHA * data.imu.acc[i];
275             }
276         }
277         for (int i = 0; i < 3; ++i) data.imu.accLpf[i] = sLpfAcc[i];
278         data.imu.accNorm = sqrtf(sLpfAcc[0] * sLpfAcc[0] +
279             sLpfAcc[1] * sLpfAcc[1] +
280             sLpfAcc[2] * sLpfAcc[2]);
281         // 動加速度ノルム = LPF 後の加速度ノルム - キャリブ済み静止ノルム（
            ≒ 重力 1g）。

```

```

282 // 軸ごとではなくスカラーで引くので姿勢に依存しない。水平で校正してか
283 // 90 度傾けて振っても残留重力で誤検出しない（旧方式の不具合）。
284 // Calibrating 中は gravityMag=0 なので dynNorm=accNorm。負側は 0 ク
285 // ランプ。
286 float dynN = data.imu.accNorm - data.calibration.gravityMag;
287 if (dynN < 0.0f) dynN = 0.0f;
288 data.imu.dynNorm = dynN;
289 // dynAcc は accLpf の向きを保ったまま大きさを dynNorm にスケール
290 // (= 重力を「現在の加速度方向の成分」として差し引いた近似ベクトル)。
291 // 経路長の二重積分はこのベクトルを使う。Armed 中は accNorm が十分大
292 // きく
293 // 向きは振りに支配されるので近似として十分。
294 if (data.imu.accNorm > 1e-3f) {
295     const float k = dynN / data.imu.accNorm;
296     for (int i = 0; i < 3; ++i) data.imu.dynAcc[i] = sLpfAcc[i] * k;
297 } else {
298     for (int i = 0; i < 3; ++i) data.imu.dynAcc[i] = 0.0f;
299 }
300 // ナビ用の重力ベクトル推定: accLpf をさらに遅い LPF に通す。
301 // dynNorm が大きい間（振り中）は更新を凍結し、振り加速度の漏れ込みを
302 // 防ぐ。
303 // 状態によらず常時回しておくことで、Menu 突入時には収束済みになって
304 // いる
305 // (Calibrating の静止 2 秒で十分収束する)。
306 if (!sNavGravInit) {
307     for (int i = 0; i < 3; ++i) sNavGrav[i] = data.imu.accLpf[i];
308     sNavGravInit = true;
309 } else if (data.imu.dynNorm < NAV_GRAV_FREEZE_G) {
310     for (int i = 0; i < 3; ++i) {
311         sNavGrav[i] += NAV_GRAV_LPF_ALPHA * (data.imu.accLpf[i] -
312         sNavGrav[i]);
313     }
314 }
315 // Armed 中のみ dynAcc を二重積分して経路長 sPathLen を更新する。
316 // Idle 中は積分しないので、起動からのノイズが蓄積することはない。
317 if (data.conductor.state == ConductorState::Conducting &&
318     data.calibration.done && sGate == BeatGate::Armed) {
319     const uint32_t sampleMs = data.imu.sampleAtMs ? data.imu.
320     sampleAtMs : now;
321     if (sLastImuMs == 0) {
322         sLastImuMs = sampleMs;
323     } else {
324         const uint32_t dtMs = sampleMs - sLastImuMs;
325         sLastImuMs = sampleMs;
326         // 5ms 周期想定。loop 遅延で dt が飛んだサンプルは捨てる。

```

```

323         if (dtMs > 0 && dtMs <= 50) {
324             const float dt = dtMs * 0.001f;
325             for (int i = 0; i < 3; ++i) {
326                 sVel[i] += data.imu.dynAcc[i] * GRAVITY_MS2 * dt;
327             }
328             const float vNorm = sqrtf(sVel[0] * sVel[0] +
329                                     sVel[1] * sVel[1] +
330                                     sVel[2] * sVel[2]);
331             sPathLen += vNorm * dt;
332         }
333     }
334     if (data.imu.dynNorm > sArmedPeakDyn) sArmedPeakDyn = data.imu.
        dynNorm;
335     data.beat.pathLenM = sPathLen;
336 } else {
337     // Conducting でない or Idle ゲート時: 積分は走らせない。
338     // pathLen は前回確定値を 0 にしてデバッグの見え方を揃える。
339     data.beat.pathLenM = sPathLen;
340 }
341 }
342
343 // 4. 状態遷移
344 // 前ループの拍検出ブロック内で起きた遷移 (Conducting→Result) をここで拾
    い、
345 // Result のナビが動き出す前にデッドタイムを開始する。
346 noteStateTransition(data, now);
347 switch (data.conductor.state) {
348     case ConductorState::Idle:
349         // SoftAP 起動完了で Calibrating へ
350         if (data.orcNet.wifiConnected) {
351             data.conductor.state = ConductorState::Calibrating;
352             data.calibration.startMs = now;
353             data.calibration.sampleCount = 0;
354             data.calibration.accumNorm = 0.0f;
355             data.calibration.done = false;
356         }
357         break;
358
359     case ConductorState::Calibrating: {
360         if (data.imu.ready) {
361             // 軸ごとではなく生加速度のノルムを累積する。停止していれば姿
                勢に
362             // 関係なく ≒1g になり、その平均を「静止ノルム」として保持す
                る。
363             const float n = sqrtf(data.imu.acc[0] * data.imu.acc[0] +
364                                   data.imu.acc[1] * data.imu.acc[1] +
365                                   data.imu.acc[2] * data.imu.acc[2]);
366             data.calibration.accumNorm += n;

```

```

367         data.calibration.sampleCount++;
368     }
369     if (now - data.calibration.startMs >= CALIBRATION_MS) {
370         if (data.calibration.sampleCount > 0) {
371             data.calibration.gravityMag =
372                 data.calibration.accumNorm /
373                 (float)data.calibration.sampleCount;
374         } else {
375             data.calibration.gravityMag = 1.0f;    // フォールバック:
376                                                     1g
377         }
378         data.calibration.done = true;
379         // production: キャリブ完了後はまず Menu へ (モード選択)。
380         data.conductor.state = ConductorState::Menu;
381         data.game.mode = 0;                // 既定カーソル = 自由演奏
382         data.game.navCursor = 0;
383         data.game.targetBpm = 0;
384         data.game.score = 0xFF;
385         sNavGate = NavGate::Idle;
386     }
387     break;
388 }
389 case ConductorState::Conducting: {
390     // Fallback 遷移は無効化: 実機テストで IMU 瞬断により演奏が遮られ
391     // 30 秒間拍が検出されなかったらメニューに自動復帰する
392     const uint32_t lastActivity = (sLastBeatMs > 0) ? sLastBeatMs :
393         sStateEnteredMs;
394     if ((now - lastActivity) > 30000) {
395         data.conductor.state = ConductorState::Menu;
396         data.game.mode = 0;
397         data.game.navCursor = 0;
398         data.game.targetBpm = 0;
399         data.game.score = 0xFF;
400         data.game.gameBeatCount = 0;
401         data.game.scoreErrAccum = 0.0f;
402         data.game.scoreWeightAccum = 0.0f;
403         gateToIdle();
404         resetTempoTracking(data);
405     }
406     break;
407 }
408 case ConductorState::Fallback:
409     // Fallback への遷移経路を全て無効化したので到達しないが、
410     // 万一入った場合は直前の状態に即復帰する。
411     data.conductor.state = sStateBeforeFallback;

```

```

412         sNavGate = NavGate::Idle;
413         break;
414
415         // ー production: メニュー (IMU ナビでモード選択。拍検出は止まる)
416         ー
417         case ConductorState::Menu: {
418             // Fallback 遷移は無効化済み (修正3)
419             if (!inTransitionDeadtime(now) && updateNav(data, now,
420                 MENU_ITEM_COUNT)) {
421                 if (data.game.navCursor == 1) {
422                     // ゲーム開始: 目標テンポ提示・採点カウンタをリセット
423                     data.game.mode = 1;
424                     data.game.targetBpm = GAME_TARGET_BPM;
425                     data.game.gameBeatCount = 0;
426                     data.game.scoreErrAccum = 0.0f;
427                     data.game.scoreWeightAccum = 0.0f;
428                     data.game.score = 0xFF;
429                 } else {
430                     // 自由演奏
431                     data.game.mode = 0;
432                     data.game.targetBpm = 0;
433                     data.game.score = 0xFF;
434                 }
435                 // 演奏セッション開始: テンポ推定と拍番号を初期化する。
436                 // beatNo=0 に戻すと楽器側は effective = beatNo-1-
437                 // headRestBeats < 0 の
438                 // 頭休符からやり直す = 毎セッション曲頭から演奏が始まる
439                 // (ゲームの「32 拍 = かえるのうた 1 周を採点」の前提と一致さ
440                 // せる)。
441                 // 楽器側の重複排除は「bn == lastBeatNo」だけなので巻き戻しも
442                 // 受理される。
443                 // ※前セッションが 1 拍だけで終わった直後に限り、新セッショ
444                 // ンの bn=1 が
445                 // 重複と誤判定され 1 拍飲まれるが、bn=2 から自己回復するた
446                 // め許容。
447                 resetTempoTracking(data);
448                 data.beat.beatNo = 0;
449                 gateToIdle();
450                 data.conductor.state = ConductorState::Conducting;
451             }
452             break;
453         }
454
455         // ー production: 結果表示 (縦振り=決定で Menu へ戻る。拍検出は止ま
456         // る) ー
457         case ConductorState::Result: {
458             // Fallback 遷移は無効化済み (修正3)
459             if (!inTransitionDeadtime(now) && updateNav(data, now, 1)) {

```

```

452         data.conductor.state = ConductorState::Menu;
453         data.game.mode = 0;
454         data.game.navCursor = 0;
455         data.game.targetBpm = 0;
456         data.game.score = 0xFF;
457         data.game.gameBeatCount = 0;
458         data.game.scoreErrAccum = 0.0f;
459         data.game.scoreWeightAccum = 0.0f;
460     }
461     break;
462 }
463 }
464
465 // 今ループの switch 内で起きた遷移 (Menu→Conducting 等) を即時反映し、
466 // 直後の拍検出ブロックがメニュー決定の振りを 1 拍目として拾わないように
467 // する。
468 noteStateTransition(data, now);
469
470 // 2-3. 拍検出 (ゲート式ステートマシン + 早期発火)
471 // Idle: dynNorm > BEAT_DYN_THRESHOLD_G で Armed に遷移し積分を開始
472 // Armed:
473 //     [発火] sPathLen が BEAT_FIRE_PATH_M に達した瞬間に拍を発火
474 //         (Armed セッション中 1 回のみ、refractory も AND)。
475 //         振り始めから ~100ms で発火するので体感的に振りと同期する。
476 //         ※発火しても Armed は維持する。これが 1 振り=1 拍の鍵で、
477 //         「発火→即 Idle→次サンプルで dyn 高いまま再 Arm→
478 //         refractory
479 //         境界で再発火」という連続発火ループを防ぐ。
480 // [リリース] 以下のいずれかで Idle に戻して次の振りに備える
481 //     (a) dynNorm < BEAT_RELEASE_G (絶対値: 完全停止)
482 //     (b) dynNorm < peak * BEAT_RELEASE_RATIO (相対値: ピークアウト)
483 //     (c) Armed 開始から BEAT_ARMED_TIMEOUT_MS 経過 (保険)
484 // 未発火セッションでは pathOk も AND して弱い振りの早期リリースを
485 // 防ぐ
486 //     (発火済セッションは pathOk が自明なので無条件)。
487 //     minHoldOk で突入直後の単発ノイズによる即リリースも防ぐ。
488 // 遷移直後のデッドタイム中は拍検出を走らせない (ゲートは遷移検知時に
489 // Idle 化済み)。
490 if (data.conductor.state == ConductorState::Conducting && data.imu.ready
491     &&
492     !inTransitionDeadtime(now)) {
493     switch (sGate) {
494         case BeatGate::Idle:
495             if (data.imu.dynNorm > BEAT_DYN_THRESHOLD_G) {
496                 sGate = BeatGate::Armed;
497                 sArmedAtMs = now;
498                 sArmedPeakDyn = data.imu.dynNorm;
499                 sVel[0] = sVel[1] = sVel[2] = 0.0f;

```



```

495         sPathLen          = 0.0f;
496         sLastImuMs        = 0;
497         sBeatFiredInArmed = false;
498     }
499     break;
500
501     case BeatGate::Armed: {
502         const uint32_t armedFor = now - sArmedAtMs;
503         const bool pathOk      = sPathLen >= BEAT_FIRE_PATH_M;
504         const bool minHoldOk    = armedFor >= BEAT_ARMED_MIN_HOLD_MS;
505
506         // —— 早期発火: 1 Armed セッションに 1 回まで ——
507         // path 閾値 + refractory 経過で発火する。Idle には戻さず
508         //   Armed を
509         // 維持し、リリース or timeout が来るのを待つ。これにより 1
510         // 振り
511         // 中の二重発火を構造的に防ぐ。
512         if (!sBeatFiredInArmed && pathOk &&
513             (now - sLastBeatMs) >= BEAT_REFRACTORY_MS) {
514             data.beat.event      = true;
515             data.beat.beatNo     += 1;
516             data.beat.lastBeatMs = now;
517
518             // テンポ推定の段取り:
519             //   1 拍目: sLastBeatMs == 0 なので何もしない (= 1 音目
520             //             は初期値 100 BPM)
521             //   2 拍目: sBpmInit == false なので「1→2 拍目の間隔」
522             //             をそのまま採用 (簡易テンポ)
523             //   3 拍目以降: BPM_EMA_ALPHA で随時補正
524             if (sLastBeatMs != 0) {
525                 const uint32_t intervalMs = now - sLastBeatMs;
526                 if (intervalMs > 0) {
527                     const float instBpm = 60000.0f / (float)
528                                             intervalMs;
529                     if (!sBpmInit) {
530                         sBpmEma = instBpm;    // 簡易テンポ確定 (1→2
531                                             拍目)
532                         sBpmInit = true;
533                     } else {
534                         sBpmEma = (1.0f - BPM_EMA_ALPHA) * sBpmEma +
535                                     BPM_EMA_ALPHA * instBpm;    // 随時
536                                     補正
537                     }
538                     if (sBpmEma < BPM_MIN) sBpmEma = BPM_MIN;
539                     if (sBpmEma > BPM_MAX) sBpmEma = BPM_MAX;
540                     data.tempo.bpm = sBpmEma;
541                     const uint32_t periodMs = (uint32_t)(60000.0f /

```

```

535         sBpmEma);
536         data.tempo.nextBeatPredictedMs = now + periodMs;
537
538         // ー production ゲーム採点 ー
539         // 実振り間隔 intervalMs と目標拍間隔の誤差を、ガ
540         // イド強度で重み付け
541         // して累積する（ガイドが薄い区間ほど重い）。1 拍
542         // 目は基準が無いので除外。
543         if (data.game.mode == 1 && data.game.targetBpm >
544             0 &&
545             data.game.gameBeatCount >= 1) {
546             const float targetInterval = 60000.0f / (
547                 float)data.game.targetBpm;
548             const float err = fabsf((float)intervalMs -
549                 targetInterval);
550             const float w = 1.0f - gameGuideIntensity(
551                 data.game.gameBeatCount);
552             data.game.scoreErrAccum += w * err;
553             data.game.scoreWeightAccum += w;
554         }
555     }
556     sLastBeatMs = now;
557     sBeatFiredInArmed = true;
558
559     // ー production ゲーム進行：経過拍を数え、規定拍に達し
560     // たら結果へ ー
561     if (data.game.mode == 1) {
562         data.game.gameBeatCount += 1;
563         if (data.game.gameBeatCount >= GAME_LENGTH_BEATS) {
564             // 得点確定：平均誤差を 0-100 へ写像（誤差小=高得
565             // 点）
566             float s = 100.0f;
567             if (data.game.scoreWeightAccum > 0.0f && data.
568                 game.targetBpm > 0) {
569                 const float avgErr =
570                     data.game.scoreErrAccum / data.game.
571                     scoreWeightAccum;
572                 const float targetInterval = 60000.0f / (
573                     float)data.game.targetBpm;
574                 const float tol = targetInterval *
575                     GAME_SCORE_TOLERANCE_RATIO;
576                 s = 100.0f * (1.0f - avgErr / tol);
577             }
578             if (s < 0.0f) s = 0.0f;
579             if (s > 100.0f) s = 100.0f;
580             data.game.score = (uint8_t)(s + 0.5f);
581             data.conductor.state = ConductorState::Result;

```

```

570     }
571 }
572 }
573
574 // —— リリース判定（デバウンス付き） ——
575 // 発火/未発火どちらでも評価する。
576 // 発火済なら pathOk は自明なので無条件で release/timeout で
577 // 未発火なら pathOk を AND して弱い振りでの早期リリースを防
578 // 止。
579 // releaseInst を即時には採用せず、BEAT_RELEASE_HOLD_MS の間
580 // 連続して成立したらリリース確定とする。これにより、振りの
581 // 最中に dynNorm が一瞬 peak*RATIO を割るだけで Armed を抜けて
582 // しまい、再 Armed→再発火が起きるチャタリングを防ぐ。
583 const bool releaseAbs = data.imu.dynNorm < BEAT_RELEASE_G;
584 const bool releaseRatio = (sArmedPeakDyn > 0.0f) &&
585     (data.imu.dynNorm <
586      sArmedPeakDyn * BEAT_RELEASE_RATIO
587      );
588 const bool releaseInst = releaseAbs || releaseRatio;
589 if (releaseInst) {
590     if (sReleaseStartMs == 0) sReleaseStartMs = now;
591 } else {
592     sReleaseStartMs = 0;
593 }
594 const bool releaseHeld = (sReleaseStartMs != 0) &&
595     (now - sReleaseStartMs >=
596      BEAT_RELEASE_HOLD_MS);
597 const bool released = minHoldOk && releaseHeld &&
598     (sBeatFiredInArmed || pathOk);
599 const bool timeout = armedFor >= BEAT_ARMED_TIMEOUT_MS;
600 if (released || timeout) {
601     const char* exitReason = timeout ? "timeout"
602                                     : releaseAbs ? "abs"
603                                     : "ratio";
604     DBG_PRINTF("[N1_ARM_END_dur=%lu_peak=%4.2f_path=%5.3f_
605               reason=%s_fired=%s]\n",
606               (unsigned long)armedFor,
607               sArmedPeakDyn,
608               sPathLen,
609               exitReason,
610               sBeatFiredInArmed ? "YES" : "NO");
611     gateToIdle();
612     data.beat.pathLenM = 0.0f;
613 }
614 break;
615 }

```

```

612     }
613 } else if (data.conductor.state != ConductorState::Conducting) {
614     // Conducting でない状態 (Idle / Calibrating / Fallback) に入ったら
615     // ゲートを必ず Idle に戻す。
616     // ※「Conducting だが imu.ready=false」のループではここに来ない。
617     // ImuModule は 5ms 周期でサンプルするので loop の大半は ready=
        false で
618     // 通過するため、ここで gateToIdle すると Armed が次ループで毎回潰
        され、
619     // 積分が走らず ARM_END も出ない致命的バグになる。
620     if (sGate != BeatGate::Idle) gateToIdle();
621 }
622
623 // デバッグ可視化用にゲート状態を SystemData にミラー
624 data.beat.gateState = (sGate == BeatGate::Armed) ? 1 : 0;
625 data.beat.armedPeakDyn = sArmedPeakDyn;
626
627 // LED 状態
628 updateLed(data);
629 }

```

B 実装した関数のフローチャート

本文 3.1 節で述べた主要関数のうち、メインループ (図 5) 以外のフローチャートをまとめて示す。図 A.20 は指揮者ノードの拍検出・テンポ推定 (2.3 節・リスト 6)、図 A.21 は楽器ノードの受信処理と時計同期 (2.4 節・リスト 4)、図 A.22 は楽器ノードの発音予約発火と楽譜進行 (2.6 節・リスト 5) に対応する。

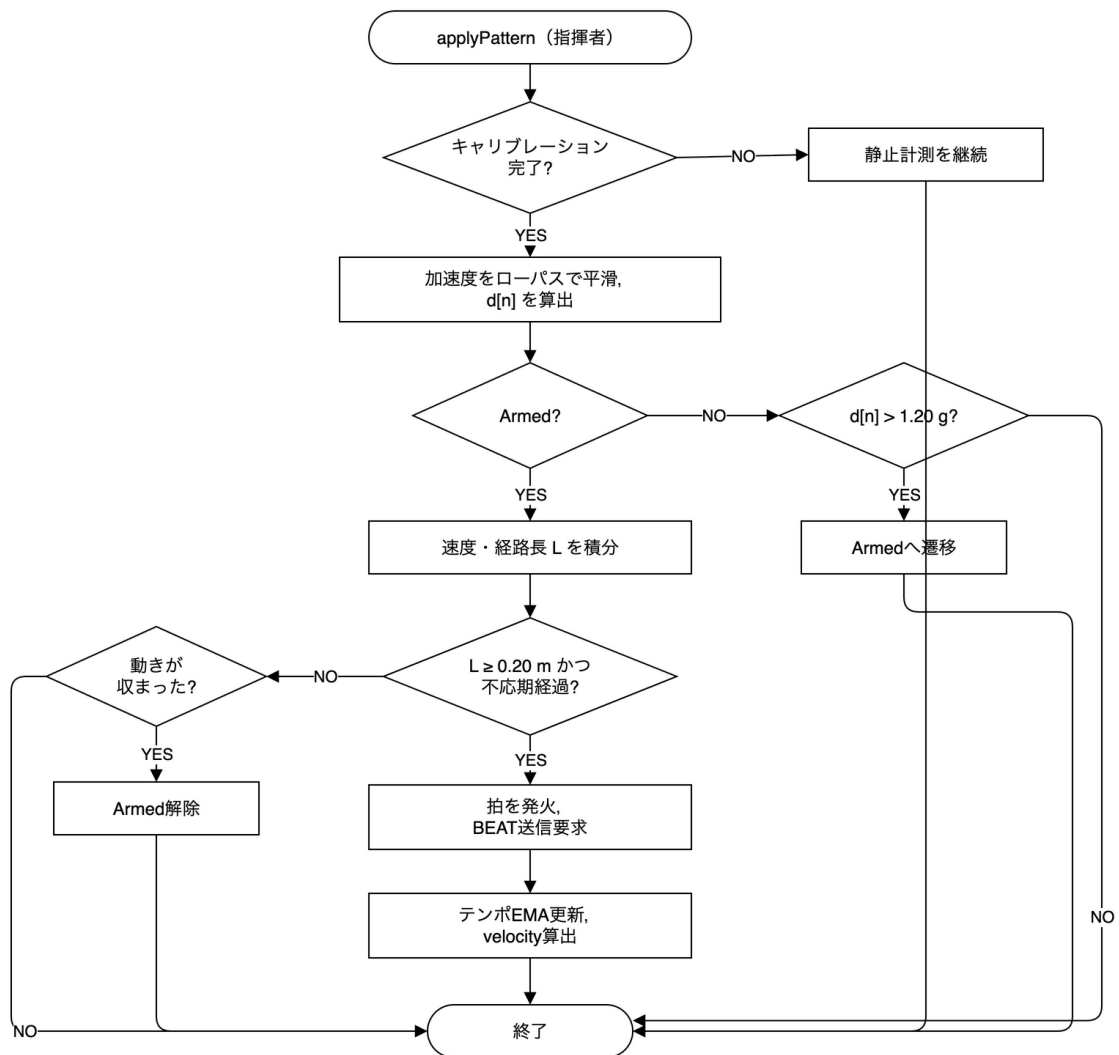


図 A.20 applyPattern() 関数（指揮者：拍検出・テンポ推定）のフローチャート

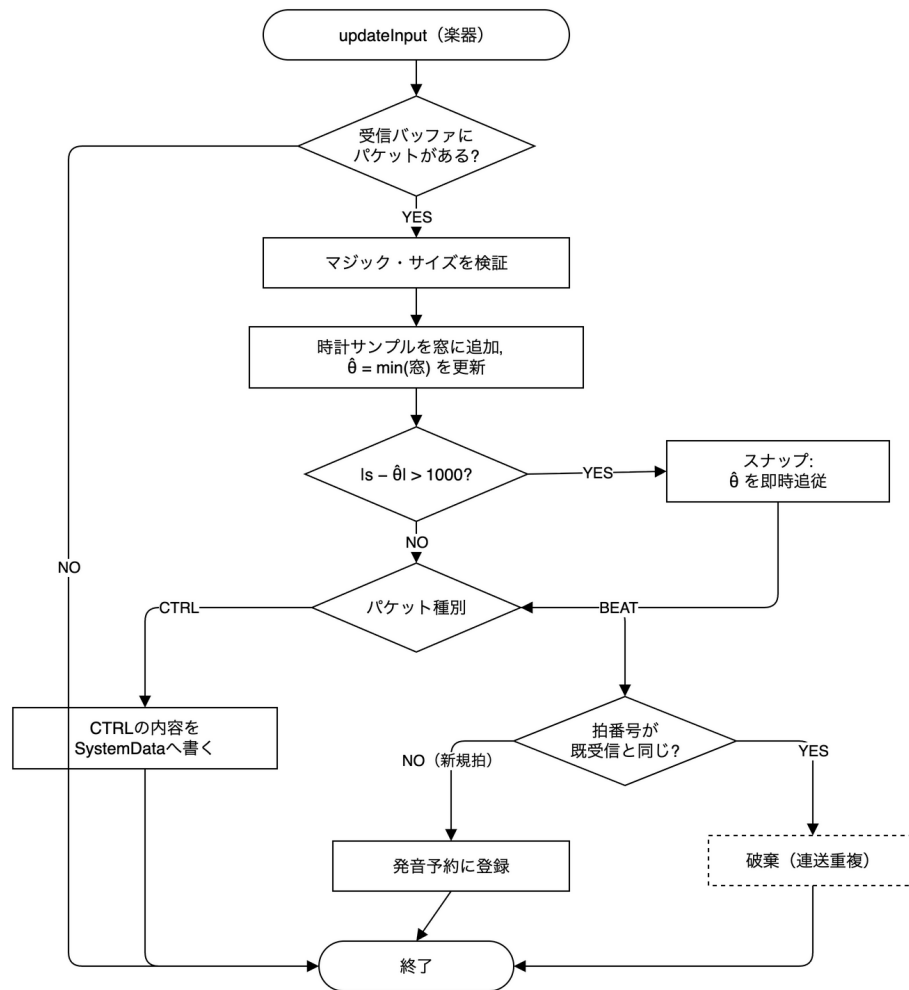


図 A.21 OrcReceiverModule::updateInput() 関数（楽器：受信・時計同期）のフローチャート

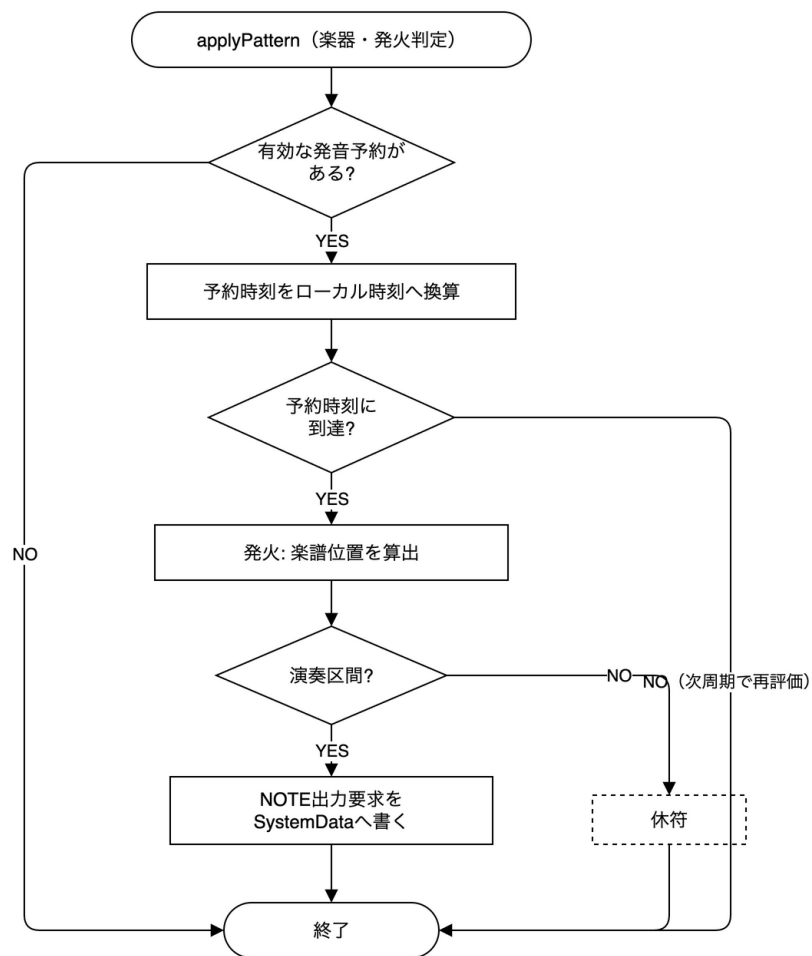


図 A.22 applyPattern() 関数（楽器：発音予約の発火・楽譜進行）のフローチャート